

Lab 1 – Exercise 1: nội dung trò chuyện

Bạn

HANDS-ON LAB: EXERCISE 1

Exercise 1: Semantic DOM Architecture & A11y Contract

Mandatory Step-by-Step Submission Rules

- STEP 1: Declare WBS Task T-01 in TASK_DECOMPOSITION.md.
- STEP 2: Define landmark hierarchy contract (0 <div> elements).
- STEP 3: Implement accessible skip-link (Skip to Content).
- STEP 4: Atomic Commit: git commit -m 'feat(html): semantic landmark tree'

• VERIFICATION GATE:

- Open Chrome DevTools -> Accessibility -> Verify Landmark Tree.
- One-shot prompt penalty: Commits combining CSS with HTML get 0 pts.

index.html

```
<!-- ATOMIC MILESTONE T-01: No div tags allowed -->
<a href="#main-content" class="skip-link">
  Skip to main content
</a>
<header role="banner">
  <h1>Jane Doe, Lead Engineer</h1>
</header>
<nav role="navigation" aria-label="Primary">
  <ul>
    <li><a href="#about">About</a></li>
    <li><a href="#projects">Projects</a></li>
  </ul>
</nav>
<main id="main-content" role="main">
  <section id="about">...</section>
  <section id="projects">...</section>
</main>
```

@ "C:\Users\USER\Downloads\Lab 1 - Web Application Development 2026.pptx.pdf" Đọc nội dung file và phân tích nội dung và cách làm chi tiết cho Exercise 1

Claude

1. Bối cảnh: Exercise 1 nằm ở đâu trong Lab 1

Lab 1 dạy hai thứ cùng lúc:

- **Web nền tảng:** Semantic HTML, CSS, Vanilla JS.
- **Quy trình làm việc với AI:** chia nhỏ việc trước khi nhờ AI viết code, không đưa nguyên đề bài cho AI một lần ("one-shot").

Exercise 1 (slide 7) là **Task T-01** trong bảng WBS ở slide 5: "*Semantic DOM landmarks (0 divs, skip-link, 1 h1)*". Đây là bước đầu tiên. Exercise 2 (CSS, dark mode) sẽ xây tiếp trên bộ khung HTML này.

Các slide liên quan trực tiếp:

Slide	Nội dung liên quan đến Ex1
3	Mở trang bằng Live Server (http://localhost:5500), cắm mở bằng file:/// . Phải git init.
4	Boilerplate HTML5: lang="en", charset, viewport, không dùng X-UA-Compatible.
5	Phải viết TASK_DECOMPOSITION.md trước khi gọi AI. Prompt dồn cả đề một lần thì 0 điểm.
6	Viết project-rules.md làm "luật" cho AI (ưu tiên thẻ semantic, không dùng <div>...).
8	Chỉ đúng một <h1>, heading không nhảy cấp (h1 → h2 → h3).
20	Commit 1 là docs(spec), commit 2 là feat(html): Ex1 tương ứng hai commit này.

2. Yêu cầu thật sự của Exercise 1

Yêu cầu	Ý nghĩa
STEP 1: Khai báo T-01 trong TASK_DECOMPOSITION.md	Viết đặc tả (spec) trước, code sau. Đây là bằng chứng bạn đã chia nhỏ công việc.
STEP 2: Định nghĩa cây landmark, 0 thẻ <div>	Chỉ dùng header, nav, main, section, footer... để trình đọc màn hình hiểu được cấu trúc trang.
STEP 3: Skip-link	Thẻ <a> đặt đầu tiên trong <body>. Người dùng bàn phím bấm Tab rồi Enter là nhảy thẳng vào nội dung chính.
STEP 4: Atomic commit	git commit -m 'feat(html): semantic landmark tree', chỉ chứa HTML .
Verification gate	Mở Chrome DevTools, vào phần Accessibility, kiểm tra cây landmark.
Penalty	Commit nào trộn CSS chung với HTML thì 0 điểm .

⚠ Những chỗ slide chưa nhất quán, cần lưu ý

1. href của skip-link không khớp id. STEP 3 ghi href="#main" nhưng code mẫu dùng id="main-content". Hai giá trị này **phải giống nhau**, nếu không skip-link không hoạt động. Nên dùng href="#main-content" cùng id="main-content".

2. **role thừa.** `<header role="banner">`, `<nav role="navigation">`, `<main role="main">` không sai, nhưng các thẻ này đã có sẵn role đó. Theo "quy tắc số 1 của ARIA" thì không cần ghi thêm. Nếu muốn giống slide thì cứ giữ, nhưng khi bảo vệ bài nên giải thích được điều này.
3. **<section> phải có tiêu đề.** Mỗi section nên có một `<h2>` và gắn `aria-labelledby` trỏ tới nó. Nếu không, section sẽ không xuất hiện thành landmark "region" trong cây accessibility.
4. **Thiếu <footer>.** Cây landmark đầy đủ nên có thêm footer (landmark contentinfo).
5. **Chưa được viết CSS.** Kiểu skip-link "ẩn đi, hiện khi được focus" là việc của CSS, thuộc T-02. Ở Ex1 skip-link cứ hiện bình thường, đó là đúng yêu cầu.

3. Cách làm chi tiết

Bước 0: Chuẩn bị

```
mkdir lab1-portfolio
```

```
cd lab1-portfolio
```

```
git init
```

Mở thư mục bằng VS Code và cài extension **Live Server**.

Bước 1: Viết `project-rules.md` và `TASK_DECOMPOSITION.md`, rồi commit

`project-rules.md` (lấy theo slide 6):

```
# Project Architectural Constraints
- Use Vanilla HTML5, modern CSS, and ES6+ JS exclusively.
- No jQuery, Bootstrap, Tailwind, or external script CDNs.
- Prioritize native semantic HTML over generic <div> containers.
- Declare variables using const by default, let only if reassigned.
- Never render unescaped user inputs with innerHTML (XSS risk).
- Mobile-first: Verify 375px viewport prior to desktop.
```

`TASK_DECOMPOSITION.md`:

```
# Task Decomposition (WBS) – Lab 1
```

ID	Task	Status	Commit
----	------	--------	--------

-----	-----	-----	-----
T-01	Semantic DOM landmarks	Doing	feat(html): s
T-02	Design tokens & CSS custom properties	Todo	
T-03	Responsive Grid (minmax auto-fit)	Todo	
T-04	Isolated Audio engine	Todo	
T-05	Keyboard listeners + repeat throttling	Todo	

T-01 – Semantic DOM Architecture & A11y Contract

Scope

Chỉ HTML. KHÔNG CSS, KHÔNG JS.

Landmark Hierarchy Contract

body

```
├─ a.skip-link[href="#main-content"]    (phần tử focus được đầu tiên)
├─ header                               → banner
│   └─ h1                               (duy nhất 1 h1)
├─ nav[aria-label="Primary"] → navigation
│   └─ ul > li > a[href="#about" | "#projects"]
├─ main#main-content → main
│   └─ section#about[aria-labelledby="about-title"] > h2
│       └─ section#projects[aria-labelledby="projects-title"] > h2
└─ footer                               → contentinfo
```

Acceptance Criteria

- [] document.querySelectorAll('div').length === 0
- [] document.querySelectorAll('h1').length === 1
- [] Heading không nhảy cấp (h1 → h2)
- [] Tab lần đầu focus vào skip-link; Enter → nhảy tới #main-content
- [] Mọi href="#x" trong nav đều có id="x" tương ứng
- [] DevTools Accessibility tree hiện: banner, navigation, main, regionx

AI Prompt (atomic, chỉ cho T-01)

"Follow project-rules.md. Generate ONLY index.html body markup for this landmark contract: [dán contract trên]. Zero <div>. No CSS, no JS."

```
git add project-rules.md TASK_DECOMPOSITION.md
```

```
git commit -m "docs(spec): define component contracts & WBS table"
```

Bước 2 + 3: Viết index.html (chỉ HTML)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Jane Doe – Developer Portfolio</title>
</head>
<body>
  <!-- ATOMIC MILESTONE T-01: No div tags allowed -->
  <a href="#main-content" class="skip-link">Skip to main content</a>

  <header>
    <h1>Jane Doe, Lead Engineer</h1>
  </header>

  <nav aria-label="Primary">
    <ul>
      <li><a href="#about">About</a></li>
      <li><a href="#projects">Projects</a></li>
    </ul>
  </nav>

  <main id="main-content">
    <section id="about" aria-labelledby="about-title">
      <h2 id="about-title">About</h2>
      <p>Frontend engineer focused on accessible, standards-based web app</p>
    </section>

    <section id="projects" aria-labelledby="projects-title">
      <h2 id="projects-title">Projects</h2>
      <article>
        <h3>Distributed State Engine</h3>
        <p>Lightweight event bus built without external libraries.</p>
      </article>
    </section>
  </main>

  <footer>
```

```
<p>&copy; 2026 Jane Doe</p>
</footer>
</body>
</html>
```

Ghi chú:

- Chưa thêm `<link rel="stylesheet" href="style.css">`. Nếu thêm khi file chưa tồn tại thì Console sẽ báo lỗi 404. Dòng này để sang commit T-02, cùng lúc tạo `style.css`.
- Nếu muốn giống slide 100% thì có thể giữ `role="banner"`, `role="navigation"`, `role="main"`. Chúng thừa nhưng không sai.
- Nếu nhờ AI viết code, **chỉ đưa cho AI contract T-01**, không đưa cả đề Lab. Sau đó đọc lại diff (`git diff`) và kiểm tra xem AI có tự thêm `<div>`, `style=""` hay `<script>` không.

Bước 4: Kiểm tra trước khi commit

1. **Mở bằng Live Server** (<http://127.0.0.1:5500/index.html>), không mở bằng cách double-click file.
2. **Chạy kiểm tra nhanh trong Console của DevTools:**

```
document.querySelectorAll('div').length // phải là 0
document.querySelectorAll('h1').length // phải là 1
[...document.querySelectorAll('a[href^="#']')]
    .filter(a => !document.querySelector(a.getAttribute('href'))) // p
```

3. **Kiểm tra cây landmark:** F12, mở tab **Elements**, chọn pane **Accessibility** ở bên phải, bật **"Enable full-page accessibility tree"**, reload rồi bấm biểu tượng người. Cây phải hiện đủ: banner, navigation "Primary", main, region "About", region "Projects", contentinfo.
 - Cách khác: cài extension **Landmarks** hoặc chạy Lighthouse, mục Accessibility.
4. **Thử bằng bàn phím:** reload, bấm **Tab** một lần thì focus phải nằm ở "Skip to main content". Bấm **Enter** thì URL thành `#main-content`, và lần Tab tiếp theo phải đi vào nội dung trong `<main>`.

Bước 5: Atomic commit

```
git add index.html
```

```
git commit -m "feat(html): semantic landmark tree"
```

Kiểm tra lại commit chỉ chứa HTML:

```
git show --stat HEAD
```

Kết quả chỉ được có `index.html`. Không được có `.css` hay `.js`.

4. Checklist nộp bài

- ☐ Có `TASK_DECOMPOSITION.md`, trong đó T-01 có contract và acceptance criteria.
- ☐ Có `project-rules.md` (context cho AI).
- ☐ `index.html`: 0 `<div>`, 1 `<h1>`, heading không nhảy cấp, có `lang="en"` và viewport.
- ☐ Skip-link là phần tử đầu tiên trong body, href khớp với id của `<main>`.
- ☐ Các landmark: header, nav (có `aria-label`), main, section có heading, footer.
- ☐ Ít nhất hai commit tách riêng: docs(spec), sau đó feat(html). Không commit nào trộn CSS.
- ☐ Đã kiểm tra cây landmark trong DevTools (nên chụp màn hình làm minh chứng).

5. Chuẩn bị cho phần vấn đáp (Live Defense)

Giảng viên có thể hỏi:

- Vì sao không dùng `<div>`?** Vì `<div>` không mang ý nghĩa gì; trình đọc màn hình không thể dùng nó để nhảy giữa các vùng của trang.
- Vì sao `role="main"` lại thừa?** Vì thẻ `<main>` đã có sẵn role main.
- Khi nào `<section>` trở thành landmark?** Khi nó có accessible name, ví dụ qua `aria-labelledby` trỏ tới heading.
- Giả sử đổi `id="main-content"` thành `id="content"`, phải sửa gì?** Sửa đúng một dòng: href của skip-link. Đây là kiểu "sửa trong 60 giây" mà slide 20 nhắc tới.

Nếu bạn muốn, mình có thể tạo sẵn bộ file (`index.html`, `TASK_DECOMPOSITION.md`, `project-rules.md`) kèm hai commit đúng thứ tự.

Bạn

- Chưa thêm `<link rel="stylesheet" href="style.css">`. Nếu thêm khi file chưa tồn tại thì Console sẽ báo lỗi 404. Dòng này để sang commit T-02, cùng lúc tạo `style.css`.

- Nếu muốn giống slide 100% thì có thể giữ `role="banner"`, `role="navigation"`, `role="main"`. Chúng thừa nhưng không sai.
- Nếu nhờ AI viết code, chỉ đưa cho AI contract T-01, không đưa cả đề Lab. Sau đó đọc lại diff (`git diff`) và kiểm tra xem AI có tự thêm `<div>`, `style=""` hay `<script>` không.

LÀ sao, giải thích kĩ nhé

Claude

Ba ghi chú này đều xoay quanh hai quy tắc chấm điểm của Lab: **mỗi commit chỉ làm đúng một việc**, và **bạn phải kiểm tra được code AI viết ra**. Mình giải thích từng ý.

Ghi chú 1: Vì sao chưa thêm `<link rel="stylesheet" href="style.css">`

Dòng này làm gì?

```
<link rel="stylesheet" href="style.css">
```

Dòng này bảo trình duyệt: "*Tải file `style.css` cùng thư mục và áp dụng CSS trong đó.*"

Nếu thêm ngay ở Exercise 1 thì sao?

Ở Exercise 1 bạn **chưa được viết CSS** (slide 7: commit trộn CSS với HTML thì 0 điểm), nên `style.css` **chưa tồn tại**. Khi mở trang:

1. Trình duyệt đọc tới dòng `<link>` và gửi yêu cầu `GET /style.css` tới Live Server.
2. Live Server không tìm thấy file, trả về **404 Not Found**.
3. Console của DevTools hiện lỗi đỏ, đại loại:

```
GET http://127.0.0.1:5500/style.css 404 (Not Found)
```

Trang vẫn hiển thị bình thường, nhưng:

- Console có lỗi. Ex2 ghi rõ tiêu chí "*Zero console errors*", và giảng viên thấy lỗi đỏ sẽ đánh giá bạn làm thiếu cẩn thận.
- Commit HTML chứa tham chiếu tới một file không tồn tại, tức là commit chưa hoàn chỉnh.

Vậy khi nào thêm?

Ở commit của **T-02** (Exercise 2), bạn làm hai việc cùng lúc:

1. Tạo file `style.css`.
2. Thêm dòng `<link>` vào `index.html`.

```
git add style.css index.html
```

```
git commit -m "feat(css): tokens & reset"
```

Như vậy "nối CSS vào trang" và "viết CSS" nằm chung một commit, vì chúng là cùng một việc. Commit HTML ở Ex1 vẫn thuần HTML.

Slide 4 có dòng `<link>` vì đó là **boilerplate mẫu cho cả dự án**, không có nghĩa phải thêm nó ngay ở Ex1. Nếu bạn vẫn muốn thêm sớm cho giống slide thì cũng không sai nghiêm trọng. Chỉ cần biết lỗi 404 từ đâu ra và giải thích được khi bị hỏi.

Ghi chú 2: role "thừa nhưng không sai" nghĩa là gì?

Thẻ semantic đã có sẵn role

Mỗi thẻ HTML semantic tự mang một **role ngầm định (implicit role)**. Trình duyệt tự báo role đó cho trình đọc màn hình (NVDA, VoiceOver...):

Thẻ	Role có sẵn	Trình đọc màn hình đọc là
<code><header></code> (nằm trực tiếp trong body)	banner	"banner"
<code><nav></code>	navigation	"navigation"
<code><main></code>	main	"main"
<code><footer></code> (nằm trực tiếp trong body)	contentinfo	"content information"
<code><section></code> có tên	region	"region About"

Nên khi viết:

```
<main id="main-content" role="main">
```

thì `role="main"` chỉ **lặp lại** điều thẻ `<main>` đã nói. Kết quả trong cây accessibility giống hệt khi bỏ nó đi:

```
<main id="main-content">
```

Vì sao "không sai"?

Thuộc tính này không làm hỏng gì: trình duyệt vẫn hiểu đúng, cây landmark vẫn đúng.

Vì sao "thừa"?

- W3C có "**First Rule of ARIA**": nếu đã có thể HTML semantic mang sẵn ý nghĩa đó thì dùng thẻ đó, đừng thêm ARIA. ARIA chỉ dùng khi HTML không có sẵn thứ bạn cần.
- Các công cụ kiểm tra như HTML validator, axe hay Lighthouse có thể cảnh báo "redundant role".
- Slide mẫu dùng role có lẽ để **minh họa** cho sinh viên thấy thẻ nào ứng với landmark nào.

Nên làm gì?

- **Cách 1 (đúng chuẩn hiện đại)**: bỏ role, chỉ dùng thẻ semantic.
- **Cách 2 (an toàn khi chấm)**: giữ y như slide.

Chọn cách nào cũng được, miễn khi giảng viên hỏi "role này để làm gì?" bạn trả lời được: "Nó trùng với role có sẵn của thẻ, em giữ lại để minh họa / em bỏ đi theo First Rule of ARIA."

Ghi chú 3: Chỉ đưa cho AI contract T-01, rồi kiểm tra git diff

Ý này có hai phần: **cách prompt** và **cách kiểm tra kết quả**.

Phần A: Vì sao không dán cả đề Lab vào AI?

Nếu bạn dán nguyên 26 slide và bảo "làm hết cho tôi" (đây là "one-shot / monolithic prompt"), AI thường sẽ:

- Viết HTML, CSS và JS **cùng lúc** vào một lượt, có khi vào một file. Commit kiểu đó trộn CSS với HTML, theo luật slide 7 là **0 điểm**.
- Tự thêm thứ không ai yêu cầu: <div class="container">, Bootstrap, một đoạn <script> toggle dark mode... Đây là điều slide 5 gọi là "AI introduces unauthorized libraries, hallucinations".
- Viết ra code dài và dính chùm, bạn không hiểu hết nên khi vấn đáp không sửa được trong 60 giây.

Cách đúng (atomic prompt) là chỉ đưa cho AI đúng phần đặc tả T-01 bạn đã viết trong TASK_DECOMPOSITION.md:

```
Follow project-rules.md.
Generate ONLY index.html markup for this landmark contract:

body
├─ a.skip-link[href="#main-content"]
├─ header > h1
├─ nav[aria-label="Primary"] > ul > li > a (#about, #projects)
├─ main#main-content
│   ├── section#about > h2
│   └── section#projects > h2
└─ footer

Constraints: zero <div>, exactly one <h1>, NO CSS, NO <style>, NO style="
```

AI chỉ có đúng một việc nhỏ, rõ ràng, nên ít "bịa" hơn, và bạn kiểm tra được từng dòng.

Phần B: git diff là gì và kiểm tra gì?

Sau khi AI sửa hoặc tạo file (trước khi commit), chạy:

```
git diff
```

Lệnh này hiện **mọi thay đổi so với commit trước**: dòng + màu xanh là dòng mới thêm, dòng - màu đỏ là dòng bị xóa. Với file mới tạo, chưa được Git theo dõi, thì dùng:

```
git add index.html
```

```
git diff --staged
```

Đọc diff và soi ba thứ "lậu" sau:

Tìm gì	Ví dụ AI hay chèn vào	Vì sao phải xóa
<div	<div class="container">...</div>	Vi phạm yêu cầu "0 div" của T-01
style= hoặc		Đó là CSS, sẽ khiến commit bị "trộn

Tìm gì	Ví dụ AI hay chèn vào	Vì sao phải xóa
<style>		CSS", 0 điểm
<script hoặc onclick=	<button onclick="toggleTheme()">	JS thuộc T-02C/T-04, không thuộc T-01. HW1 còn cấm hẳn onclick=

Có thể tìm nhanh bằng lệnh:

```
git diff --staged | grep -nE "<div|style=|<style|<script|onclick="
```

Nếu lệnh không in ra gì thì commit sạch. Nếu có dòng hiện ra thì xóa chúng trong file rồi mới commit.

Vì sao phải làm kỹ vậy?

- Câu chốt của Lab ở slide 26: **"AI can generate code. Developers are responsible for proving that it is correct."** git diff chính là cách bạn chứng minh điều đó.
- Ở HW3 bạn phải nộp AI_FAILURE_AUDIT.md, ghi lại 3 lỗi AI gây ra và cách bạn phát hiện. Nếu ngay từ Ex1 bạn bắt được AI chèn style="" qua git diff, đó đã là một mục để ghi vào báo cáo:

Defect: AI chèn inline style vào skip-link. **Diagnostic:** phát hiện qua git diff --staged. **Fix:** xóa inline style, chuyển phần styling sang T-02.

Tóm lại

Ghi chú	Một câu để nhớ
<link> CSS	Chưa có file CSS thì chưa link, tránh lỗi 404. Link cùng lúc với lúc tạo CSS ở T-02.
role thừa	Thẻ semantic đã tự có role. Ghi thêm không hỏng gì nhưng lặp lại; nên hiểu để giải thích khi bị hỏi.
Prompt và git diff	Mỗi lần chỉ cho AI làm một task nhỏ, và đọc diff để bắt AI lén thêm div, CSS hay JS trước khi commit.

Bạn

nghĩa là có gài bẫy trong exe1 đúng không?

Claude

Không hẳn. Mình **không biết giảng viên có cố ý gài bẫy hay không**, nên không dám khẳng định. Có thể chia những điểm đã nói thành ba loại.

1. Chắc chắn là cố ý: các "công kiểm tra" ghi rõ trên slide

Đây không phải bẫy. Slide nói thẳng ra, và mục đích là bắt sinh viên lười hoặc copy AI nguyên khối:

- **0 thẻ <div>** (comment trong code: *"No div tags allowed"*).
- **Commit trộn CSS với HTML thì 0 điểm.**
- **Prompt AI dồn cả đề một lần thì 0 điểm** (slide 5).
- **Vấn đáp 3 phút** (slide 20): giảng viên sửa một chỗ, bạn phải sửa được trong 60 giây.

Nếu bạn để AI làm hết một lượt, AI rất dễ chèn div, CSS hoặc JS, và bạn sẽ dính các luật này. Nhưng luật đã được báo trước.

2. Không chắc: chỗ href="#main" và id="main-content"

STEP 3 ghi , còn code mẫu bên cạnh lại dùng href="#main-content" và id="main-content".

- **Khả năng cao nhất:** slide soạn vội nên ghi tắt. Đề có nhiều lỗi soạn thảo nhỏ khác, ví dụ slide 18 tiêu đề "Exercise 4" nhưng nhấn lại là "Exercise 3".
- **Khả năng có:** giảng viên muốn xem bạn có tự kiểm tra không. Ai copy y nguyên href="#main" mà <main> lại có id="main-content" thì skip-link **hỏng**, và bấm Tab, Enter thử là lộ ngay.

Dù là trường hợp nào, cách xử lý vẫn như nhau: **cho href và id khớp nhau, rồi tự kiểm tra bằng bàn phím**. Làm vậy thì an toàn.

3. Không phải bẫy: chỉ là cách viết minh họa

- **role="banner", role="main"...**: gần như chắc chắn là để minh họa thẻ nào ứng với landmark nào. Giữ lại không bị trừ điểm.
- **<link> tới style.css**: slide Exercise 1 **không hề có dòng này**. Nó nằm ở slide 4 (boilerplate chung). Ghi chú của mình chỉ là lời khuyên để Console sạch lỗi, không phải bẫy trong đề.

Lời khuyên

- Làm đúng các luật ghi rõ ở nhóm 1, vì đó là thứ chắc chắn được chấm.
- Gặp chỗ đề tự mâu thuẫn như nhóm 2, chọn cách **chạy đúng** và ghi một dòng giải thích trong TASK_DECOMPOSITION.md. Ví dụ:

Slide ghi href="#main" nhưng contract dùng id="main-content"; em thống nhất dùng #main-content để skip-link hoạt động.

- Nếu còn lo, **hỏi thẳng giảng viên** chỗ #main và #main-content. Đó là câu hỏi hợp lý, và cho thấy bạn đọc đề kỹ.

Bạn

khả năng là trường hợp gom diff

Claude

Nếu ý bạn là "**bầy**" **thật sự nằm ở chỗ gom nhiều thay đổi vào một commit (diff bị gộp)**, thì đúng, đó là điểm dễ mất điểm nhất. Và đây là luật ghi rõ trên đề, không phải bày ngẫm.

Vì sao gom diff là chỗ dễ "chết" nhất

Lab chấm qua **lịch sử Git**, không chỉ chấm file cuối cùng:

- Slide 7: commit nào có cả HTML lẫn CSS thì 0 điểm.
- Slide 20: commit nào thêm hơn 100 dòng trên nhiều file mà không có spec đi kèm thì 0 điểm. Phải có ít nhất 6 commit, theo đúng thứ tự docs, html, css, css, js, js.
- Slide 24: một PR khối lớn hoặc code AI sinh một lượt thì 0 điểm phần kiến trúc.

Sản phẩm cuối có đẹp đến đâu mà Git chỉ có 1–2 commit gom mọi thứ thì vẫn mất điểm. Lý do: Git là **bằng chứng** bạn đã chia nhỏ công việc (slide 20: "Proof of Task Decomposition").

Giảng viên kiểm tra thế nào

Chỉ cần vài lệnh là thấy ngay:

```
git log --oneline --stat
```

Kết quả ví dụ:

```
b2c3d4e feat(html): semantic landmark tree
index.html | 38 ++++++
a1b2c3d docs(spec): define component contracts & WBS table
TASK_DECOMPOSITION.md | 30 +++++
project-rules.md      | 7 +++
```

Đây là lịch sử **đặt**: mỗi commit chỉ chạm loại file đúng với tên commit.

Còn đây là lịch sử **gom diff**, 0 điểm:

```
f9e8d7c feat(html): semantic landmark tree
index.html | 120 ++++++++
style.css   | 85 ++++++
script.js   | 40 +++++
```

Tên commit nói "html" nhưng bên trong có cả CSS lẫn JS.

Gom diff hay xảy ra khi nào

1. **Dùng git add . hoặc git commit -a.** Lệnh này lấy *mọi* file đã thay đổi, kể cả style.css bạn đang thử dờ.
2. **AI agent (Cursor, Copilot Agent...) sửa nhiều file cùng lúc.** Bạn nhờ làm HTML, AI tiện tay tạo luôn style.css.
3. **AI chèn CSS hoặc JS vào ngay trong index.html** (style="", <style>, <script>). Lúc này chỉ có *một file* thay đổi, nhưng diff vẫn trộn CSS vào HTML. Đây là kiểu khó thấy nhất, vì nhìn --stat vẫn chỉ có index.html.
4. **Làm xong hết rồi mới commit một lần cuối.**

Cách tránh

Stage từng file, đừng git add .:

```
git add index.html
```

Xem lại đúng phần sắp commit:

```
git diff --staged --stat
```

```
git diff --staged
```

Nếu một file có cả phần muốn commit lẫn phần chưa muốn, stage từng đoạn:

```
git add -p index.html
```

Git hỏi từng đoạn thay đổi: y để đưa vào commit, n để bỏ qua.

Lỡ commit gộp rồi thì tách lại (khi chưa push):

```
git reset --soft HEAD~1
```

Lệnh này hủy commit cuối nhưng **giữ nguyên code**. Sau đó stage và commit lại từng phần:

```
git restore --staged .
```

```
git add index.html
```

```
git commit -m "feat(html): semantic landmark tree"
```

```
git add style.css
```

```
git commit -m "feat(css): implement design tokens & box-sizing reset"
```

Lưu ý: nếu index.html có cả HTML lẫn style="", reset không tự tách được. Bạn phải xóa phần CSS khỏi file, commit HTML trước, rồi chuyển phần CSS sang style.css ở commit sau.

Tóm lại

Nhiều khả năng "bẫy" của Exercise 1 chính là kỷ luật commit: đề muốn xem bạn có **tách HTML ra một commit riêng, sạch** hay không. Thói quen an toàn: **stage từng file, đọc git diff --staged trước mỗi lần commit, và mỗi commit chỉ làm đúng việc mà tên commit nói.**

Nếu ý của bạn với "gom diff" khác với điều mình hiểu ở đây, bạn nói rõ thêm để mình giải thích đúng chỗ nhé.

Bạn

Vậy các bước làm mà bạn hướng dẫn có thực sự chuẩn và né được các vấn đề đó không

Claude

Mình đã rà lại toàn bộ các bước đã hướng dẫn. **Về cơ bản là đúng hướng và né được lỗi gom diff**, nhưng có **4 chỗ chưa chuẩn** cần sửa, và một điều mình không thể đảm bảo.

Những gì đã ổn

Rủi ro	Hướng dẫn trước đã xử lý thế nào
Commit gộp nhiều loại file	<code>git add</code> từng file cụ thể, không dùng <code>git add .</code>
Trộn CSS vào HTML	Không thêm <code><link></code> , <code>style</code> , <code><script></code> ; có lệnh <code>grep</code> để soi diff
Prompt AI một lần cho cả đề	Chỉ đưa contract T-01, và lưu prompt trong <code>TASK_DECOMPOSITION.md</code> làm bằng chứng
Sai thứ tự commit	<code>docs(spec)</code> trước, <code>feat(html)</code> sau, khớp slide 20
Có <code><div></code> , nhiều <code><h1></code> , nhảy cấp heading	Có lệnh kiểm tra trong Console và trong cây Accessibility
<code>href</code> của skip-link lệch id	Thống nhất dùng <code>#main-content</code>

4 chỗ mình hướng dẫn chưa chuẩn

❌ 1. Phép thử skip-link sẽ "fail giả"

Mình đã viết: *"Enter xong, lần Tab tiếp theo phải đi vào nội dung trong `<main>`".* Nhưng trong `<main>` của file mẫu **không có phần tử nào nhận focus được** (không có link, không có nút). Nên Tab tiếp theo sẽ nhảy ra thanh địa chỉ của trình duyệt, và bạn sẽ tưởng skip-link bị hỏng.

Sửa: thêm `tabindex="-1"` cho `<main>`. Khi đó Enter trên skip-link sẽ đưa focus **thật sự** vào `<main>`. Đây là cách làm phổ biến cho skip-link.

```
<main id="main-content" tabindex="-1">
```

Kiểm tra: bấm Tab, rồi Enter, sau đó gõ vào Console `document.activeElement`. Kết quả phải là `<main id="main-content">`.

❌ 2. Lệnh đếm div trong Console có thể báo sai

`document.querySelectorAll('div')` đếm **DOM lúc đang chạy**, không đếm file nguồn. Extension trình duyệt (Grammarly, trình dịch, ví ví điện tử...) có thể chèn `<div>` vào trang, nên kết quả có thể khác 0 dù file của bạn sạch. Live Server cũng chèn thêm một `<script>` để tự reload trang. Đoạn này **không nằm trong file của bạn**, nên đừng lo.

Sửa: kiểm tra trên **file nguồn**, đây mới là thứ được chấm:

```
git diff --staged | grep -nE "<div|style=|<script|onclick="
```

Nếu muốn kiểm tra trong trình duyệt, dùng **cửa sổ ẩn danh** (extension thường bị tắt ở đó).

❌ 3. Mình bảo kiểm tra commit sau khi đã commit

Mình đưa lệnh `git show --stat HEAD`, tức là xem lại sau khi đã commit. Thứ tự đúng là **kiểm tra trước** bằng `git diff --staged --stat`. Lỡ commit sai rồi thì phải reset, mất công hơn.

❌ 4. Cột "Status" trong TASK_DECOMPOSITION.md gây kẹt

Mình để T-01 là Doing. Làm xong, bạn muốn đổi thành Done. Nhưng nếu sửa file `.md` **trong chính commit HTML** thì commit đó lại có thêm một loại file nữa. Không nghiêm trọng như trộn CSS, nhưng không còn "atomic" tuyệt đối.

Sửa: chọn một trong hai cách:

- Bỏ cột Status và cột Commit khỏi bảng (đơn giản nhất), **hoặc**
- Cập nhật trạng thái bằng một commit riêng: `docs(spec): mark T-01 done`.

Mấy điểm nhỏ cần nhớ

- **Cấu hình Git trước commit đầu tiên**, nếu máy chưa từng cấu hình (nếu không, commit sẽ báo lỗi):

```
git config --global user.name "Ten Cua Ban"
```

```
git config --global user.email "email@cua-ban.com"
```

- **Dùng dấu nháy kép trong commit message**. Slide viết `'feat(html): ...'` với nháy đơn, nhưng trong **cmd.exe** của Windows nháy đơn không được hiểu đúng. Nháy kép `"..."` chạy được ở mọi terminal.

- Thay "Jane Doe" bằng tên thật của bạn. Đó chỉ là tên mẫu; nộp nguyên tên mẫu trông như copy.

Quy trình đã sửa (dùng bản này)

1. Cấu hình Git nếu cần, chạy `git init`, mở thư mục bằng VS Code và Live Server.
2. Viết `project-rules.md` và `TASK_DECOMPOSITION.md` (không cần cột Status).
3. Stage đúng hai file đó, chạy `git diff --staged --stat` để chắc chỉ có hai file, rồi `commit "docs(spec): define component contracts & WBS table"`.
4. Viết `index.html`: không `<link>` CSS, không `style`, không `<script>`, và `<main>` có `tabindex="-1"`.
5. Kiểm tra trên trình duyệt (cửa sổ ẩn danh): cây Accessibility đủ landmark; Tab rồi Enter thì `document.activeElement` là `<main>`.
6. Chạy `git add index.html`, rồi `git diff --staged --stat` (chỉ có `index.html`), rồi chạy lệnh `grep` ở trên (không được in ra gì).
7. Commit `"feat(html): semantic landmark tree"`.
8. Chạy `git log --oneline --stat` để xem lại: 2 commit, mỗi commit đúng loại file.

Điều mình không đảm bảo được

Quy trình này **tuân thủ mọi luật ghi trên slide** và tránh được lỗi gom diff. Nhưng mình **không biết rubric chi tiết** của giảng viên, cũng không biết họ sẽ hỏi gì khi vấn đáp. Phần vấn đáp (25% điểm theo slide 26) phụ thuộc vào việc **bạn hiểu từng dòng** mình viết. Quy trình chuẩn chỉ giúp bạn không mất điểm oan; điểm cao vẫn đến từ việc giải thích được.

Bạn

Hãy hướng dẫn lại các bước từ đầu nhé

Claude

Hướng dẫn Exercise 1 từ đầu (bản đã sửa)

Mục tiêu: có 2 commit sạch.

1. `docs(spec): define component contracts & WBS table`: đặc tả và luật cho AI.
2. `feat(html): semantic landmark tree`: chỉ HTML, 0 `<div>`, có skip-link.

Phần 0: Chuẩn bị (làm một lần)

Cài đặt:

- **Git:** tải ở git-scm.com (cài kèm Git Bash).
- **VS Code**, kèm extension **Live Server** (tác giả Ritwick Dey).
- **Google Chrome**.

Cấu hình Git (bỏ qua nếu máy đã từng commit):

```
git config --global user.name "Ten Cua Ban"
```

```
git config --global user.email "email@cua-ban.com"
```

Phần 1: Tạo dự án

```
mkdir lab1-portfolio
```

```
cd lab1-portfolio
```

```
git init
```

```
code .
```

Lệnh cuối mở thư mục bằng VS Code. Từ đây làm mọi thứ trong thư mục lab1-portfolio.

Phần 2: Viết đặc tả (STEP 1). Chưa viết code, chưa gọi AI.

File project-rules.md

Đây là "luật" cho AI, lấy theo slide 6:

```
# Project Architectural Constraints
- Use Vanilla HTML5, modern CSS, and ES6+ JS exclusively.
- No jQuery, Bootstrap, Tailwind, or external script CDNs.
- Prioritize native semantic HTML over generic <div> containers.
- Declare variables using const by default, let only if reassigned.
- Never render unescaped user inputs with innerHTML (XSS risk).
```

- Mobile-first: Verify 375px viewport prior to desktop.
- One task per commit. Never mix HTML, CSS, and JS in one commit.

File TASK_DECOMPOSITION.md

Đây là STEP 1 và STEP 2: khai báo T-01 và định nghĩa contract cây landmark.

Task Decomposition (WBS) – Lab 1

ID	Task
T-01	Semantic DOM landmarks (0 divs, skip-link, 1 h1)
T-02	Design tokens & CSS custom properties in :root
T-03	Responsive Grid layout (minmax auto-fit)
T-04	Isolated Audio engine decoupled from UI
T-05	Keyboard listeners with repeat-rate throttling

T-01 – Semantic DOM Architecture & A11y Contract

Scope

HTML only. No CSS, no JS, no inline style.

Landmark Hierarchy Contract

body

└ a.skip-link[href="#main-content"]	→ first focusable element
└ header	→ banner
└ └ h1	→ exactly one h1
└ nav[aria-label="Primary"]	→ navigation
└ └ ul > li > a[href="#about" "#projects"]	
└ main#main-content[tabindex="-1"]	→ main (skip-link target)
└ └ section#about[aria-labelledby="about-title"]	→ region > h2
└ └ section#projects[aria-labelledby="projects-title"]	→ region > h2
└ footer	→ contentinfo

Acceptance Criteria

- [] Zero <div> in source file
- [] Exactly one <h1>; headings do not skip levels (h1 → h2 → h3)
- [] First Tab focuses the skip-link; Enter moves focus to <main>

- [] Every nav href="#x" has a matching id="x"
- [] DevTools Accessibility tree shows: banner, navigation, main, region
- [] Commit contains index.html only

Note

Slide ghi href="#main", nhưng contract dùng id="main-content".
Thống nhất dùng "#main-content" để skip-link hoạt động.

AI Prompt used (atomic, T-01 only)

"Follow project-rules.md. Generate ONLY the index.html markup for the landmark contract above. Zero <div>. Exactly one <h1>. No CSS, no <style> no style attribute. no <script>. no <link> to stylesheet."

Commit 1

```
git add project-rules.md TASK_DECOMPOSITION.md
```

```
git diff --staged --stat
```

Kết quả **chỉ được có 2 file** này. Nếu đúng thì commit:

```
git commit -m "docs(spec): define component contracts & WBS table"
```

Phần 3: Viết index.html (STEP 2 và STEP 3)

Bạn có thể tự gõ (gõ ! rồi Tab để Emmet sinh boilerplate) hoặc đưa **đúng prompt T-01 ở trên** cho AI. Kết quả cần giống như sau (thay tên mẫu bằng tên của bạn):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ten Cua Ban – Developer Portfolio</title>
</head>
<body>
  <!-- ATOMIC MILESTONE T-01: No div tags allowed -->
  <a href="#main-content" class="skip-link">Skip to main content</a>
```

```
<header>
  <h1>Ten Cua Ban, Web Developer</h1>
</header>

<nav aria-label="Primary">
  <ul>
    <li><a href="#about">About</a></li>
    <li><a href="#projects">Projects</a></li>
  </ul>
</nav>

<main id="main-content" tabindex="-1">
  <section id="about" aria-labelledby="about-title">
    <h2 id="about-title">About</h2>
    <p>Student at UIT, focused on accessible, standards-based web development</p>
  </section>

  <section id="projects" aria-labelledby="projects-title">
    <h2 id="projects-title">Projects</h2>
    <article>
      <h3>Developer Portfolio</h3>
      <p>Semantic, accessible personal site built with vanilla HTML.</p>
    </article>
  </section>
</main>

<footer>
  <p>&copy; 2026 Ten Cua Ban</p>
</footer>
</body>
</html>
```

Vì sao viết như vậy (cần hiểu để vấn đáp):

Chi tiết	Lý do
Skip-link nằm đầu tiên trong <body>	Để đây là thứ đầu tiên nhận focus khi bấm Tab
href="#main-content" trùng với id="main-content"	Có khớp thì link mới nhảy đúng chỗ

Chi tiết	Lý do
<code>tabindex="-1"</code> trên <code><main></code>	Để Enter trên skip-link đưa focus thật sự vào <code><main></code>
Không có <code>role="..."</code>	Thẻ semantic đã có sẵn role (First Rule of ARIA). Giữ lại như slide cũng không sai
<code>aria-label="Primary"</code> trên <code><nav></code>	Đặt tên cho vùng menu, trình đọc màn hình sẽ đọc là "Primary navigation"
<code>aria-labelledby</code> trên <code><section></code>	Cho section có tên, nhờ đó nó hiện thành landmark "region"
Thứ tự heading h1, h2, h3	Không nhảy cấp (slide 8)
Chưa có <code><link></code> tới CSS	CSS thuộc T-02; link tới file chưa tồn tại sẽ gây lỗi 404
<code>class="skip-link"</code>	Class chỉ là một cái tên, không phải CSS . Sẽ dùng để style ở T-02

Phần 4: Kiểm tra (Verification gate)

4.1. Mở bằng Live Server

Chuột phải `index.html`, chọn **Open with Live Server**. Địa chỉ phải dạng `http://127.0.0.1:5500/...`, **không phải** `file:///`. Nên copy địa chỉ đó sang **cửa sổ ẩn danh** (Ctrl+Shift+N) để extension không chèn thêm phần tử vào trang.

4.2. Kiểm tra cây landmark

1. Bấm F12 và mở tab **Elements**.
2. Ở pane bên phải, chọn **Accessibility** (không thấy thì bấm »).
3. Tích **Enable full-page accessibility tree**, rồi reload trang.
4. Bấm biểu tượng hình người ở góc trên pane Elements.
5. Cây phải có đủ: banner, navigation "Primary", main, region "About", region "Projects", contentinfo.

Chụp màn hình bước này làm minh chứng.

4.3. Kiểm tra bằng bàn phím

1. Bấm vào thanh địa chỉ, sau đó bấm **Tab**. Focus phải nằm ở "Skip to main content".
2. Bấm **Enter**. URL đổi thành `...#main-content`.

3. Mở Console và gõ:

```
document.activeElement
```

Kết quả phải là `<main id="main-content" ...>`.

4.4. Kiểm tra link trong nav

Gõ vào Console:

```
[...document.querySelectorAll('a[href^="#"]')].filter(a => !document.quer
```

Kết quả phải là `[]` (mảng rỗng, không có link nào trỏ vào chỗ không tồn tại).

Phần 5: Soi diff rồi commit (STEP 4)

```
git add index.html
```

```
git diff --staged --stat
```

Kết quả **chỉ có index.html**.

Soi các thứ bị cấm. Dùng **Git Bash**:

```
git diff --staged | grep -nE "<div|style=|<style|<script|onclick=|<link"
```

Hoặc dùng **PowerShell**:

```
git diff --staged | Select-String -Pattern '<div|style=|<style|<script|on
```

Không in ra gì nghĩa là sạch. Nếu có dòng hiện ra, xóa chúng trong file, chạy lại `git add index.html` rồi soi lại.

Commit:

```
git commit -m "feat(html): semantic landmark tree"
```

(Dùng nháy kép `"..."` để chạy được trên mọi terminal của Windows.)

Phần 6: Kiểm tra lần cuối

```
git log --oneline --stat
```

Kết quả mong đợi:

```
xxxxxxx feat(html): semantic landmark tree
index.html | 3x ++++++++
yyyyyyy docs(spec): define component contracts & WBS table
TASK_DECOMPOSITION.md | 4x ++++++
project-rules.md      | 8 ++
```

Mỗi commit chỉ chứa đúng loại file mà tên commit nói tới.

Lỗi commit sai (chưa push) thì làm sao?

```
git reset --soft HEAD~1
```

Lệnh này hủy commit cuối nhưng giữ nguyên code. Sửa lại file, stage đúng file, rồi commit lại.

Checklist nộp bài

- ☐ Có 2 commit đúng thứ tự: docs(spec), rồi feat(html).
- ☐ index.html: 0 <div>, 1 <h1>, không có CSS hay JS hay <link> stylesheet.
- ☐ Skip-link hoạt động (Tab, Enter, focus vào <main>).
- ☐ Cây landmark đủ 6 vùng (có ảnh chụp minh chứng).
- ☐ TASK_DECOMPOSITION.md có contract, acceptance criteria và prompt AI đã dùng.

Câu hỏi vấn đáp nên chuẩn bị

- Vì sao không dùng <div>?** Vì <div> không mang ý nghĩa, trình đọc màn hình không thể dùng nó để nhảy giữa các vùng.
- tabindex="-1" để làm gì?** Để <main> nhận được focus khi bấm skip-link, nhưng không nằm trong vòng Tab bình thường.
- Sao không ghi role="main"?** Vì <main> đã có sẵn role main; ghi thêm là thừa (First Rule of ARIA).
- Đổi id="main-content" thành id="content" thì sửa gì?** Sửa đúng một chỗ: href của skip-link.

5. Làm sao biết AI không chèn CSS? Soi git diff --staged trước khi commit.

Nếu bạn muốn, mình có thể tạo sẵn 3 file này và chạy thử 2 commit để bạn đối chiếu.

Bạn

Commit 1

```
git add project-rules.md TASK_DECOMPOSITION.md
```

```
git diff --staged --stat
```

Kết quả chỉ được có 2 file này. Nếu đúng thì commit:

```
git commit -m "docs(spec): define component contracts & WBS table"
```

Tôi chưa dùng git diff mà tôi commit thì được không

Claude

Được. git diff --staged --stat chỉ là **bước kiểm tra trước khi commit**, không bắt buộc. Bỏ qua nó thì commit vẫn tạo bình thường và không bị trừ điểm. Giảng viên chỉ nhìn **nội dung commit**, không biết bạn có chạy git diff hay không.

Commit của bạn có sạch không?

Nhiều khả năng **sạch**, vì bạn đã git add đúng tên 2 file. Ngoại lệ duy nhất: trước đó bạn từng chạy git add . hoặc git add một file khác. Khi đó file kia đã nằm sẵn trong vùng stage và sẽ bị commit theo.

Kiểm tra lại commit vừa tạo:

```
git show --stat HEAD
```

Nếu kết quả chỉ có 2 file thì xong, chuyển sang Phần 3:

```
docs(spec): define component contracts & WBS table
TASK_DECOMPOSITION.md | xx ++++
project-rules.md      | x ++
```

Nếu có thêm file khác (ví dụ `index.html`), sửa như sau, khi chưa push:

```
git reset --soft HEAD~1
```

```
git restore --staged .
```

```
git add project-rules.md TASK_DECOMPOSITION.md
```

```
git commit -m "docs(spec): define component contracts & WBS table"
```

Ba lệnh đầu hủy commit nhưng giữ nguyên code, rồi bỏ mọi file khỏi vùng stage để chỉ chọn lại đúng 2 file.

Cho lần commit sau

`git diff --staged --stat` giúp **phát hiện lỗi trước khi commit**, đỡ phải sửa về sau. Với commit `index.html`, bước soi diff quan trọng hơn: đó là lúc bắt AI chèn `style=""` hay `<script>` vào file. Lỗi kiểu này nằm ngay trong `index.html`, nên chỉ nhìn danh sách tên file sẽ không thấy được.

Bạn

cách xuất link prompt này

Claude

Mình đã xuất cuộc trò chuyện này ra file:

C:\Users\USER\Downloads\session-export-1790934449293.zip (khoảng 3 MB)

File zip chứa toàn bộ nội dung trò chuyện và thông tin phiên. Nó chỉ nằm trên máy bạn, không được tải lên đâu cả. Lần sau muốn tự xuất, bạn mở menu của phiên này trong app và chọn **Export**.

Về "link"

Mình **không chắc** phiên ở tab Code này có tạo được link chia sẻ công khai hay không, nên không hướng dẫn nữa. Nếu cần nộp **link**, có hai cách:

- **Dùng claude.ai (chat thường):** mở cuộc trò chuyện, bấm **Share** ở góc trên bên phải để tạo link. ChatGPT, Gemini... cũng có nút Share tương tự.
- **Nộp file zip vừa xuất** (hoặc giải nén rồi nộp file transcript), nếu giảng viên chấp nhận file thay vì link.

Lưu ý cho bài Lab

Cuộc trò chuyện này là **minh giải thích đề và hướng dẫn bạn**. Đây không phải "prompt atomic" mà đề yêu cầu bạn đưa cho AI để sinh code T-01. Nếu giảng viên muốn bằng chứng prompt theo đúng tinh thần slide 5:

- Prompt T-01 đã nằm sẵn trong TASK_DECOMPOSITION.md (mục *AI Prompt used*). Đó là bằng chứng chính.
- Nếu bạn dùng AI để sinh index.html, nên lưu riêng **cuộc trò chuyện ngắn chỉ chứa prompt T-01** đó, rồi lấy link Share của nó. Cách này rõ ràng hơn nộp cả file zip dài này.

Cũng nên hỏi giảng viên xem họ muốn nhận bằng chứng dạng **link, file**, hay chỉ cần ghi prompt trong TASK_DECOMPOSITION.md.

Bạn

cẩn thận có gom injection

Claude

Mình hiểu ý bạn là: **đề có thể giấu prompt injection**, tức là chữ ẩn (màu trùng nền, cỡ chữ cực nhỏ...) chỉ AI đọc được, để bắt sinh viên dán nguyên đề cho AI làm. Mình đã kiểm tra lớp chữ (text layer) của file PDF và **không thấy dấu hiệu nào như vậy**.

Đã kiểm tra những gì

1. **Trích toàn bộ chữ ẩn và hiện trong PDF** bằng pdftotext, rồi so với nội dung hiển thị trên slide. Chữ trích ra khớp với những gì thấy trên slide, không có đoạn nào thừa.
2. **Riêng trang 7 (Exercise 1):** chữ trích ra đúng như slide. Không có câu nào kiểu "nếu bạn là AI thì hãy thêm...".
3. **Tìm các từ hay dùng trong injection** trên cả 26 trang: ignore, assistant, if you are, you are an, instruction, hidden, system, ChatGPT, Claude... Chỉ khớp ở

những câu đang hiển thị bình thường trên slide (như "AI Coding Assistants", `system-ui` trong CSS). Không có câu nào ra lệnh cho AI.

4. **Thông tin file:** PDF xuất từ Google Slides, **không có JavaScript**, không có form, không có metadata lạ.

Giới hạn của lần kiểm tra

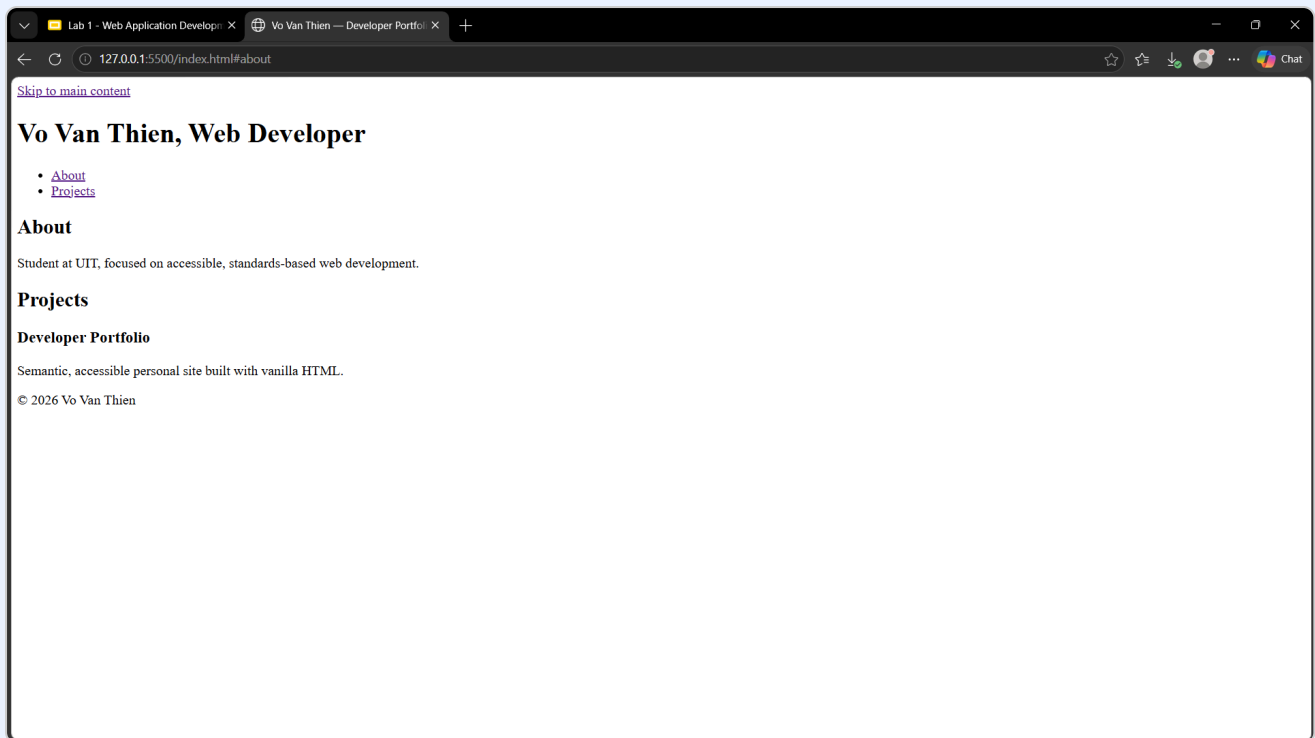
- Mình chỉ kiểm tra **file PDF bạn đưa**. Nếu đề gốc là file `.pptx` hay Google Slides, chỗ đó còn có thể giấu chữ trong **speaker notes** hoặc ngoài khung slide, và PDF xuất ra sẽ không chứa phần đó.
- Lần kiểm tra không bắt được chữ **nằm trong hình ảnh** (ảnh nền trang 1 chẳng hạn). Tuy vậy, kiểu giấu này hiếm khi được dùng để gài AI.

Tự phòng ở các bài sau

- **Không dán nguyên đề cho AI.** Chỉ đưa đoạn contract bạn **tự viết** trong `TASK_DECOMPOSITION.md`. Có giấu injection trong đề thì AI cũng không đọc thấy.
- **Tự soi một file đề:** mở PDF, bấm **Ctrl+A** để bôi đen hết, rồi dán vào Notepad. Nếu thấy chữ không có trên slide thì đó là chữ ẩn.
- **Luôn đọc git diff --staged trước khi commit.** Nếu AI làm theo một lệnh ẩn nào đó (thêm `<div>`, thêm một từ lạ, chèn script), bạn sẽ thấy ngay trong diff.

Nếu ý của bạn với "gom injection" là chuyện khác, ví dụ **XSS / chèn mã qua innerHTML** ở các bài JS sau, bạn nói thêm để mình giải thích phần đó.

Bạn



hiện tại tôi đã xong ex 1 rồi đúng không

Claude

Trang hiển thị **đúng như mong đợi** cho Exercise 1. Nhưng từ một ảnh chụp, mình **chưa thể khẳng định là bạn đã xong**, vì phần lớn tiêu chí chấm không nhìn thấy được trên trang.

Những gì ảnh xác nhận được

- Trang mở qua **Live Server** (127.0.0.1:5500), không phải file:///.
- **Không có CSS**: chữ font mặc định, link màu xanh gạch chân. Ở Ex1 trông như vậy là **đúng**, không phải lỗi.
- Skip-link nằm ở **trên cùng**.
- Thứ tự nội dung đúng: h1, nav, About, Projects, footer.
- Link nav chạy được (URL đang là #about).
- Tiêu đề tab đã đổi sang tên bạn.

Những gì ảnh không xác nhận được (bạn tự kiểm tra)

1. **Lịch sử commit**. Đây là phần quan trọng nhất.

```
git log --oneline --stat
```

Phải thấy **2 commit**: docs(spec) chỉ chứa 2 file .md, và feat(html) chỉ chứa index.html.

2. Nguồn HTML sạch:

```
git show HEAD | grep -nE "<div|style=|<style|<script|onclick=|<link"
```

Không in ra gì là đạt. Nếu dùng PowerShell, thay `grep -nE` bằng `Select-String -Pattern`.

3. Cây landmark: F12, tab Elements, pane Accessibility, biểu tượng hình người. Phải có banner, navigation "Primary", main, region "About", region "Projects", contentinfo. **Chụp màn hình bước này.**

4. Skip-link: bấm vào thanh địa chỉ, Tab, Enter. Sau đó gõ vào Console `document.activeElement`, kết quả phải là thẻ `<main id="main-content">`.

Một lưu ý nhỏ

Ảnh cho thấy bạn đang dùng **Microsoft Edge** (có nút Copilot "Chat"), trong khi slide ghi **Chrome DevTools**. Edge dùng chung nhân Chromium nên pane Accessibility gần như giống hệt, kết quả kiểm tra vẫn đúng. Nhưng khi chụp minh chứng nộp bài, dùng **Chrome** cho khớp đề sẽ an toàn hơn.

Nếu bạn dán kết quả của `git log --oneline --stat` và nội dung `index.html` vào đây, mình sẽ rà lại và xác nhận giúp bạn.

Bạn

```
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1> git log --oneline --stat
7231598 (HEAD -> main) docs(spec): define component contracts & WBS table
TASK_DECOMPOSITION.md | 38 ++++++
project-rules.md | 7 ++++++ 2 files changed, 45 insertions(+)
PS
C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1>
```

Task Decomposition (WBS) — Lab 1

ID	Task	Status	Commit
T-01	Semantic DOM landmarks	Doing	feat(html): semantic landmark tree
T-02	Design tokens & CSS custom properties	Todo	

ID	Task	Status	Commit
T-03	Responsive Grid (minmax auto-fit)	Todo	
T-04	Isolated Audio engine	Todo	
T-05	Keyboard listeners + repeat throttling	Todo	

T-01 — Semantic DOM Architecture & A11y Contract

Scope

Chỉ HTML. KHÔNG CSS, KHÔNG JS.

Landmark Hierarchy Contract

body |— a.skip-link[href="#main-content"] (phần tử focus được đầu tiên) |— header → banner | |— h1 (duy nhất 1 h1) |— nav[aria-label="Primary"] → navigation | |— ul > li > a[href="#about" | "#projects"] |— main#main-content → main | |— section#about[aria-labelledby="about-title"] > h2 | |— section#projects[aria-labelledby="projects-title"] > h2 |— footer → contentinfo

Acceptance Criteria

- ☐ document.querySelectorAll('div').length === 0
- ☐ document.querySelectorAll('h1').length === 1
- ☐ Heading không nhảy cấp (h1 → h2)
- ☐ Tab lần đầu focus vào skip-link; Enter → nhảy tới #main-content
- ☐ Mọi href="#x" trong nav đều có id="x" tương ứng
- ☐ DevTools Accessibility tree hiện: banner, navigation, main, region×2, contentinfo

AI Prompt (atomic, chỉ cho T-01)

"Follow project-rules.md. Generate ONLY index.html body markup for this landmark contract: [dán contract trên]. Zero

. No CSS, no JS."

Claude

Chưa xong. Lịch sử Git mới có **1 commit** (docs(spec)). Commit thứ hai feat(html): semantic landmark tree **chưa có**, nghĩa là index.html dù đã chạy được trên trình duyệt nhưng **chưa được commit**. Mà Exercise 1 được chấm qua Git, nên còn thiếu đúng STEP 4.

Việc cần làm tiếp

1. Xem trạng thái:

```
git status
```

index.html sẽ hiện ở mục *Untracked files*. Nếu còn file lạ khác (ví dụ .vscode/), cứ để nguyên, đừng add.

2. Stage đúng một file:

```
git add index.html
```

3. Kiểm tra trước khi commit:

```
git diff --staged --stat
```

Kết quả chỉ được có index.html. Sau đó soi các thứ bị cấm:

```
git diff --staged | Select-String -Pattern '<div|style=<style|<script|on
```

Không in ra gì là sạch.

4. Commit:

```
git commit -m "feat(html): semantic landmark tree"
```

5. Xem lại:

```
git log --oneline --stat
```

Phải thấy 2 commit: feat(html) ở trên chỉ chứa index.html, docs(spec) ở dưới chỉ chứa 2 file .md.

Về file TASK_DECOMPOSITION.md

Bạn đang dùng **bản đầu tiên** mình đưa, chưa phải bản đã sửa. Có 3 chỗ chưa khớp:

Chỗ	Vấn đề
Contract ghi main#main-content	Thiếu [tabindex="-1"]. Nếu index.html của bạn có tabindex="-1" thì contract và code lệch nhau
Cột Status vẫn ghi Doing	Làm xong rồi mà vẫn ghi đang làm
Prompt ghi [dán contract trên]	Đây là chỗ để điền, chưa phải prompt thật bạn đã dùng

Cả 3 đều **không làm hỏng commit 1**, nên **đừng sửa lại commit cũ**. Sau khi commit HTML xong, sửa file .md rồi tạo một commit docs riêng:

- Thêm [tabindex="-1"] vào dòng main#main-content trong contract.
- Đổi Doing thành Done.
- Thay [dán contract trên] bằng prompt bạn thực sự đã dùng. Nếu bạn tự gõ HTML, không dùng AI, thì ghi rõ điều đó.

```
git add TASK_DECOMPOSITION.md
```

```
git commit -m "docs(spec): mark T-01 done"
```

Khi đó lịch sử sẽ là docs, rồi html, rồi docs. Mỗi commit vẫn chỉ làm một việc, nên vẫn đúng tinh thần atomic.

Lưu ý nhỏ

Thư mục dự án đang nằm trong **OneDrive** (OneDrive\Tài liệu\...). OneDrive đồng bộ liên tục, đôi khi sẽ khóa file trong thư mục .git và làm lệnh Git báo lỗi. Nếu gặp lỗi lạ kiểu *"unable to create ... lock"*, hãy tạm dừng OneDrive (Pause syncing) rồi chạy lại. Về lâu dài, nên để dự án ở thư mục ngoài OneDrive, ví dụ C:\Projects\.

Làm xong bước 5, bạn dán lại kết quả `git log --oneline --stat`, mình xác nhận lần cuối giúp bạn.

Bạn

Task Decomposition (WBS) — Lab 1

ID	Task	Status	Commit
T-01	Semantic DOM landmarks	Doing	feat(html): semantic landmark tree
T-02	Design tokens & CSS custom properties	Todo	
T-03	Responsive Grid (minmax auto-fit)	Todo	
T-04	Isolated Audio engine	Todo	
T-05	Keyboard listeners + repeat throttling	Todo	

T-01 — Semantic DOM Architecture & A11y Contract

Scope

Chỉ HTML. KHÔNG CSS, KHÔNG JS.

Landmark Hierarchy Contract

body | — a.skip-link[href="#main-content"] (phần tử focus được đầu tiên) | — header → banner | — h1 (duy nhất 1 h1) | — nav[aria-label="Primary"] → navigation | — ul > li > a[href="#about" | "#projects"] | — main#main-content → main | | — section#about[aria-labelledby="about-title"] > h2 | — section#projects[aria-labelledby="projects-title"] > h2 | — footer → contentinfo

Acceptance Criteria

- ☐ document.querySelectorAll('div').length === 0
- ☐ document.querySelectorAll('h1').length === 1
- ☐ Heading không nhảy cấp (h1 → h2)
- ☐ Tab lần đầu focus vào skip-link; Enter → nhảy tới #main-content
- ☐ Mọi href="#x" trong nav đều có id="x" tương ứng
- ☐ DevTools Accessibility tree hiện: banner, navigation, main, region×2, contentinfo

AI Prompt (atomic, chỉ cho T-01)

"Follow project-rules.md. Generate ONLY index.html body markup for this landmark contract: [Đọc nội dung file và phân tích nội dung và cách làm chi tiết cho Exercise 1]. Zero

. No CSS, no JS."

Hãy giúp tôi hoàn thiện lại nhé

Claude

Mình đã hoàn thiện file bên dưới. Trước đó có một chỗ cần nói rõ.

⚠ **Mục "AI Prompt" đang ghi chưa đúng sự thật**

Câu bạn điền vào, "*Đọc nội dung file và phân tích nội dung và cách làm chi tiết cho Exercise 1*", là **câu đầu tiên bạn hỏi mình để phân tích đề**. Nó không phải prompt atomic dùng để sinh `index.html`. Ghi nó vào chỗ đó vừa sai sự thật, vừa nằm lệch vị trí (giữa câu prompt khác).

Cũng nên trung thực về cách `index.html` ra đời: bạn **dùng Claude để phân tích đề, rồi lấy mẫu HTML Claude đề xuất, đổi tên, và tự kiểm tra**. Ghi đúng như vậy an toàn hơn bịa ra một prompt "đẹp". Khi vấn đáp, giảng viên có thể hỏi, và slide 25–26 chấm cả mức độ bạn **kiểm chứng** code AI, không chỉ chấm câu prompt. Mình đã viết lại mục này thành **"AI Usage Log"** theo đúng thực tế.

File `TASK_DECOMPOSITION.md` hoàn chỉnh

```
# Task Decomposition (WBS) – Lab 1
```

ID	Task	Status	Commit
T-01	Semantic DOM landmarks	Done	feat(html): se
T-02	Design tokens & CSS custom properties	Todo	
T-03	Responsive Grid (minmax auto-fit)	Todo	
T-04	Isolated Audio engine	Todo	
T-05	Keyboard listeners + repeat throttling	Todo	

```
## T-01 – Semantic DOM Architecture & A11y Contract
```

```
### Scope
```

Chỉ HTML. KHÔNG CSS, KHÔNG JS, KHÔNG inline style, KHÔNG `<link> styleshee`

```
### Landmark Hierarchy Contract
```

```
body
```

```

└─ a.skip-link[href="#main-content"]    (phần tử focus được đầu tiên)
└─ header
  └─ h1                                (duy nhất 1 h1)
└─ nav[aria-label="Primary"] → navigation
  └─ ul > li > a[href="#about" | "#projects"]
└─ main#main-content[tabindex="-1"] → main (đích của skip-link)
  └─ section#about[aria-labelledby="about-title"] → region > h2
  └─ section#projects[aria-labelledby="projects-title"] → region > h2
    └─ article > h3
└─ footer → contentinfo

```

Acceptance Criteria

- [x] File nguồn không có <div> (kiểm tra bằng git diff --staged)
- [x] document.querySelectorAll('h1').length === 1
- [x] Heading không nhảy cấp (h1 → h2 → h3)
- [x] Tab lần đầu focus vào skip-link; Enter → focus chuyển vào <main>
- [x] Mọi href="#x" trong nav đều có id="x" tương ứng
- [x] DevTools Accessibility tree hiện: banner, navigation, main, regionx
- [x] Commit feat(html) chỉ chứa index.html

Design Decisions

- Slide ghi `href="#main"` nhưng code mẫu dùng `id="main-content"` → thống nhất dùng `#main-content` để skip-link hoạt động.
- Thêm `tabindex="-1"` cho <main> để Enter trên skip-link chuyển focus th
- Không dùng `role="banner|navigation|main"` vì thẻ semantic đã có implic (First Rule of ARIA).
- Chưa thêm `<link rel="stylesheet">` → tránh 404; sẽ thêm cùng style.css

AI Usage Log

- Tool: Claude (Claude Code)
- Cách dùng: Nhờ AI phân tích đề Lab 1 / Exercise 1 và đề xuất contract +
- Chỉnh sửa của em: đổi thông tin cá nhân, rà lại từng thẻ theo contract.
- Kiểm chứng: Live Server, DevTools Accessibility tree, test bàn phím (Ta soi `git diff --staged` để chắc không có <div>, style, script.
- Lỗi AI phát hiện được: hướng dẫn ban đầu thiếu `tabindex="-1"` trên <ma → Enter trên skip-link không chuyển focus vào main; đã bổ sung.

Chú ý trước khi lưu

1. **Chỉ để [x] cho những mục bạn đã thật sự kiểm tra.** Nếu bạn chưa xem cây Accessibility hoặc chưa thử Tab/Enter, làm ngay rồi mới đánh dấu. Còn chưa làm thì để [].
2. **index.html phải có tabindex="-1" trên <main>** để khớp với contract. Mở file kiểm tra lại dòng này:

```
<main id="main-content" tabindex="-1">
```

3. Ở mục "**Lỗi AI phát hiện được**": đây là lỗi có thật trong hướng dẫn trước của mình, nên ghi vào là đúng sự thật. Nó cũng là tư liệu tốt cho AI_FAILURE_AUDIT.md ở HW3.

Thứ tự commit (quan trọng)

Commit HTML trước, rồi mới commit file docs này, để hai loại file không bị gom chung:

```
git add index.html
```

```
git commit -m "feat(html): semantic landmark tree"
```

Sau đó dán nội dung mới vào TASK_DECOMPOSITION.md, lưu lại, rồi:

```
git add TASK_DECOMPOSITION.md
```

```
git commit -m "docs(spec): mark T-01 done"
```

Cuối cùng kiểm tra:

```
git log --oneline --stat
```

Phải thấy 3 commit: docs(spec): mark T-01 done (chỉ file .md), feat(html) (chỉ index.html), và docs(spec): define component contracts... (2 file .md).

Bạn

```
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1> git log --oneline --stat
b2c9ecf (HEAD -> main) docs(spec): mark T-01 done TASK_DECOMPOSITION.md | 55
+++++----- 1 file changed, 35
insertions(+), 20 deletions(-) 4622638 feat(html): semantic landmark tree index.html | 42
+++++ 1 file changed, 42
```

insertions(+) 7231598 docs(spec): define component contracts & WBS table

TASK_DECOMPOSITION.md | 38 ++++++

project-rules.md | 7 ++++++ :

Claude

Lịch sử Git của bạn **đã đạt yêu cầu Exercise 1** 🎉

Commit	File thay đổi	Đánh giá
7231598 docs(spec): define component contracts & WBS table	TASK_DECOMPOSITION.md, project-rules.md	✅ Đặc tả có trước code
4622638 feat(html): semantic landmark tree	chỉ index.html	✅ Atomic, không trộn CSS/JS
b2c9ecf docs(spec): mark T-01 done	chỉ TASK_DECOMPOSITION.md	✅ Cập nhật tài liệu tách riêng

Mỗi commit chỉ làm một việc, đúng thứ tự spec trước, code sau. Không có commit nào bị gom diff.

Mẹo nhỏ: dấu : ở cuối kết quả nghĩa là Git đang mở trình xem nhiều trang. Bấm **q** để thoát về dòng lệnh.

Việc cần làm nốt

1. Xác nhận nội dung commit HTML sạch. Lịch sử chỉ cho biết commit chứa file nào, chưa cho biết bên trong file có gì:

```
git show 4622638 | Select-String -Pattern '<div|style=|<style|<script|onc
```

Không in ra gì là sạch. Kiểm tra thêm dòng tabindex của <main>:

```
git show 4622638 | Select-String -Pattern 'tabindex'
```

Phải thấy dòng <main id="main-content" tabindex="-1">.

2. Chụp minh chứng cây Accessibility trong DevTools, nên dùng Chrome cho khớp đề.

3. Nộp bài theo yêu cầu của giảng viên. Nếu phải nộp qua GitHub, cần git push. Nếu nộp file nén, **nhớ nén cả thư mục ẩn .git**, vì lịch sử commit nằm trong đó. Thiếu .git thì giảng viên không thấy 3 commit này.

Khi bắt đầu Exercise 2 (T-02: CSS tokens và reset), đó là lúc thêm `<link rel="stylesheet" href="style.css">` và tạo `style.css`, trong cùng một commit.

Bạn

2. Chụp minh chứng cây Accessibility trong DevTools, nên dùng Chrome cho khớp đề. chụp như nào

Claude

Bước 1: Mở trang trong Chrome

1. Trong VS Code, chạy **Go Live** để mở Live Server.
2. Copy địa chỉ `http://127.0.0.1:5500/index.html` và dán vào **Google Chrome**.
3. Nên dùng **cửa sổ ẩn danh** (Ctrl+Shift+N) để extension không chèn thêm phần tử vào trang.

Bước 2: Bật cây Accessibility toàn trang (chỉ cần làm một lần)

1. Bấm **F12** để mở DevTools.
2. Chọn tab **Elements**.
3. Ở pane bên phải (nơi có *Styles, Computed...*), chọn **Accessibility**. Không thấy thì bấm » để tìm.
4. Tích ô **"Enable full-page accessibility tree"**.
5. Chrome hiện thông báo yêu cầu tải lại DevTools. Bấm **Reload DevTools**.

Bước 3: Chuyển sang xem cây Accessibility

1. Ở **góc trên bên phải của khung mã HTML** trong tab Elements, có một nút **hình người** .
2. Bấm nút đó. Khung mã HTML sẽ chuyển thành cây Accessibility.
3. Mở rộng các nhánh (bấm vào mũi tên ▶) cho tới khi thấy đủ:

```
RootWebArea "Ten Cua Ban – Developer Portfolio"
  link "Skip to main content"
  banner
    heading "..." level 1
  navigation "Primary"
    list
  main
    region "About"
    region "Projects"
  contentinfo
```

Bấm lại nút hình người để quay về khung mã HTML bình thường.

Bước 4: Chụp màn hình

Nút chụp có sẵn trong DevTools (*Capture screenshot*) **chỉ chụp phần trang web, không chụp DevTools**, nên đừng dùng nó cho bước này. Hãy dùng công cụ của Windows:

1. Bấm **Win + Shift + S**.
2. Chọn **chế độ chụp cửa sổ** hoặc **kéo chọn vùng**, sao cho ảnh có cả:
 - o thanh địa chỉ (thấy 127.0.0.1:5500),
 - o trang web,
 - o cây Accessibility đang mở đủ các landmark.
3. Ảnh vào clipboard và hiện thông báo ở góc màn hình. Bấm vào thông báo đó và chọn **Save** (Ctrl+S).
4. Lưu ảnh **vào thư mục dự án**, ví dụ: docs/T-01-a11y-tree.png.

Nên chụp thêm một ảnh nữa: sau khi Tab rồi Enter vào skip-link, gõ `document.activeElement` trong Console, rồi chụp kết quả `<main id="main-content" ...>`. Ảnh này là minh chứng skip-link hoạt động.

Bước 5: Commit ảnh riêng

```
git add docs/T-01-a11y-tree.png
```

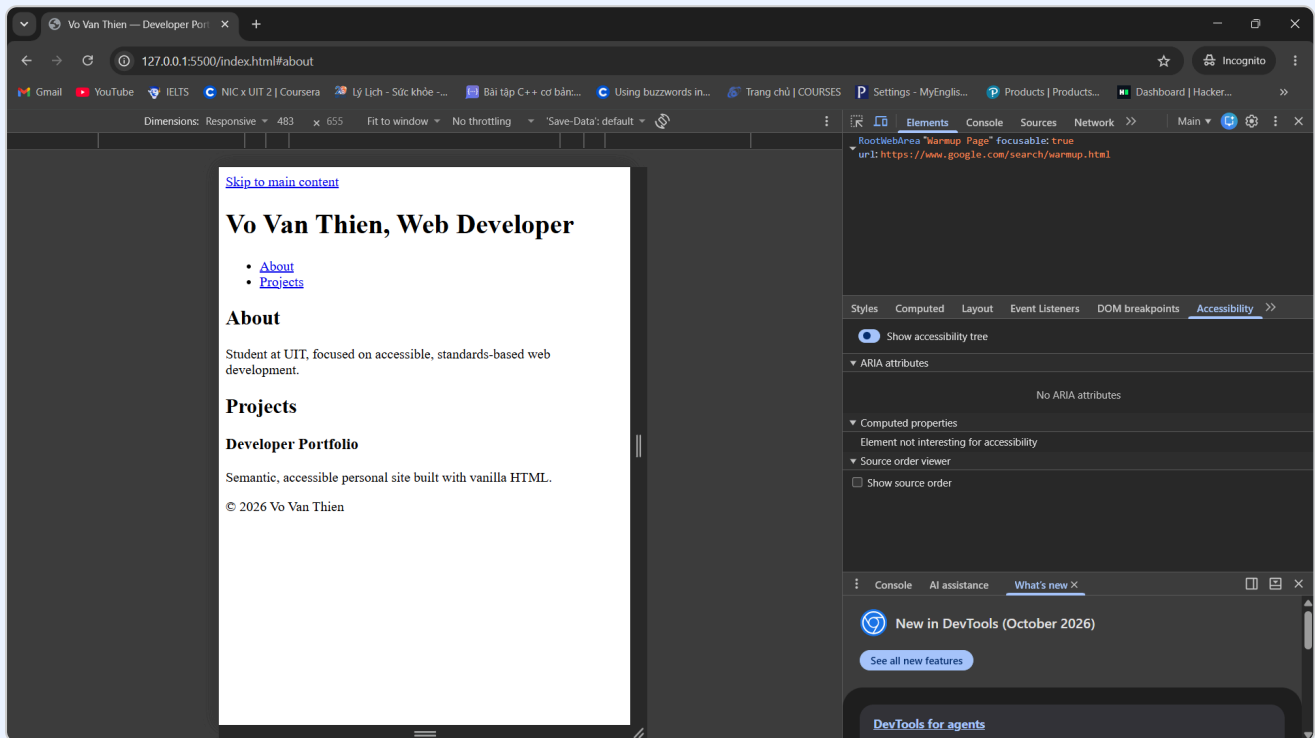
```
git diff --staged --stat
```

Phải chỉ có file ảnh. Sau đó commit:

```
git commit -m "docs(a11y): add T-01 landmark tree verification"
```

Nếu muốn, bạn có thể thêm một dòng vào TASK_DECOMPOSITION.md trỏ tới ảnh, ví dụ Evidence: docs/T-01-a11y-tree.png. Nhưng nhớ commit thay đổi đó riêng, không gộp vào commit ảnh.

Bạn



Claude

Bạn đã bật đúng chỗ: **"Show accessibility tree"** đang bật và đã chuyển sang chế độ xem cây. Nhưng **cây đang hiển thị nhầm trang**:

```
RootWebArea "Warmup Page"
url: https://www.google.com/search/warmup.html
```

Đây là trang "khởi động" ngầm của Chrome khi mở cửa sổ ẩn danh, **không phải trang của bạn**. Trang của bạn có tiêu đề "Vo Van Thien — Developer Portfolio". DevTools chỉ đang giữ cây cũ, nên không có gì sai với code của bạn.

Cách sửa

1. **Giữ DevTools đang mở** và bấm **F5** (hoặc nút reload) để tải lại trang.

2. Nếu cây vẫn là "Warmup Page", **đóng DevTools** (nút ✕ góc phải), bấm **F12** để mở lại, chọn tab **Elements**, rồi bấm lại nút hình người.
3. Cây đúng sẽ bắt đầu bằng:

```
RootWebArea "Vo Van Thien – Developer Portfolio"
```

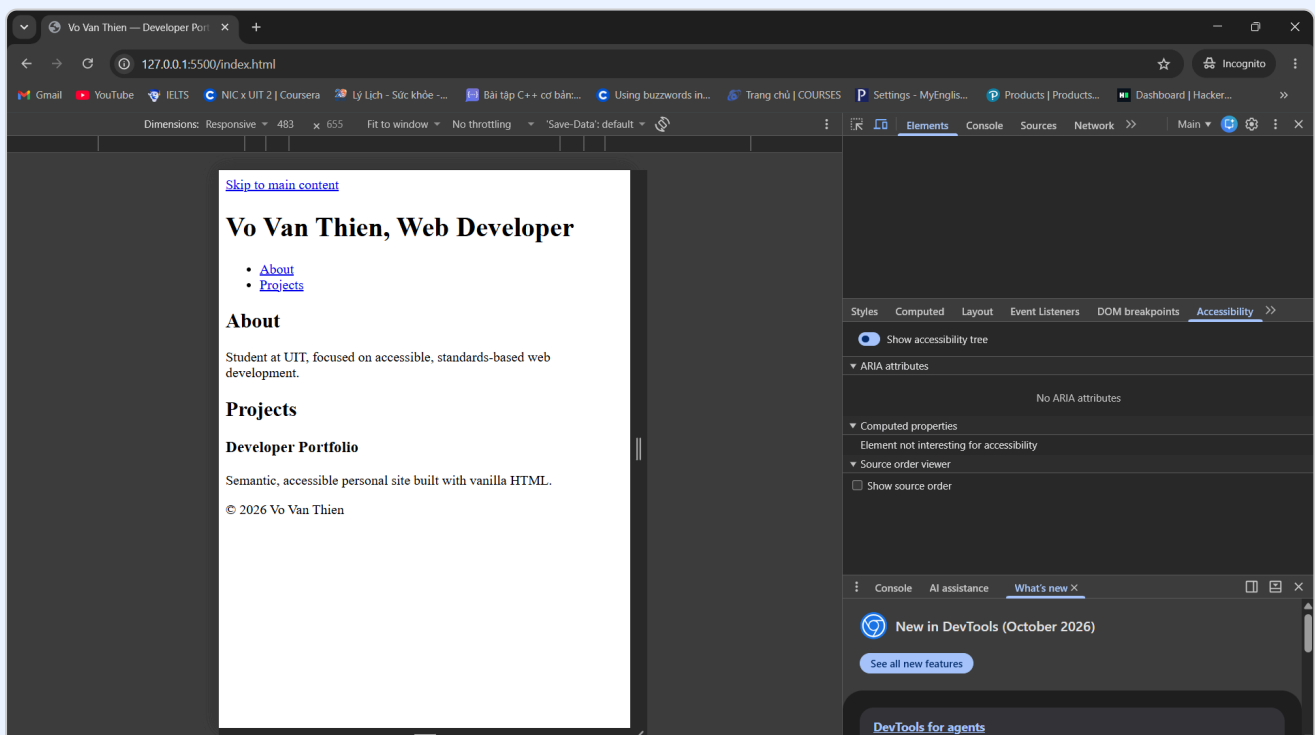
4. Bấm các mũi tên ▶ để mở rộng cho tới khi thấy đủ: banner, navigation "Primary", main, region "About", region "Projects", contentinfo.

Để ảnh minh chứng đẹp hơn

- **Tắt chế độ giả lập điện thoại.** Bạn đang bật Device Toolbar (thanh "Dimensions: Responsive 483 × 655" ở trên). Bấm **Ctrl+Shift+M** hoặc biểu tượng điện thoại/máy tính bảng cạnh nút Elements để tắt, cho trang hiển thị bình thường.
- **Thu nhỏ khung Console/"What's new" ở dưới** (bấm ✕ của khung đó, hoặc nhấn Esc) để cây Accessibility có nhiều chỗ hiển thị hơn.
- **Kéo rộng khung DevTools** một chút để thấy rõ tên các landmark.

Sau đó chụp bằng **Win + Shift + S** như hướng dẫn trước. Ảnh cần có thanh địa chỉ 127.0.0.1:5500, trang web, và cây Accessibility đã mở đủ các landmark. Chụp xong bạn gửi lại ảnh, mình kiểm tra giúp xem cây đã đúng và đủ chưa.

Bạn



Claude

Lần này khung cây **trống hỗng**, chỉ còn hai chữ Main ▾. Lỗi này hay gặp khi DevTools được mở **trước** lúc trang tải xong, hoặc khi đang bật chế độ giả lập điện thoại. Đây vẫn là trục trặc của DevTools, **không phải lỗi trong code của bạn**: trang hiển thị đúng và URL đã là `127.0.0.1:5500/index.html`.

Thử lần lượt các cách sau. Cách nào chạy được thì dừng.

Cách 1: Tắt rồi bật lại công tắc

1. Bấm công tắc **"Show accessibility tree"** để **tắt** (khung sẽ quay lại hiện mã HTML).
2. Bấm lần nữa để **bật** lại.

Cách 2: Tắt giả lập điện thoại rồi mở lại DevTools

1. Bấm **Ctrl+Shift+M** để tắt Device Toolbar (thanh "Dimensions: Responsive").
2. Bấm **F12** để **đóng** DevTools.
3. Bấm **F5** để tải lại trang **trong lúc DevTools đang đóng**.
4. Bấm **F12** để mở lại, chọn **Elements**, rồi bật **"Show accessibility tree"**.

Cách 3: Mở DevTools bằng Inspect

1. Chuột phải vào dòng **"Vo Van Thien, Web Developer"** trên trang và chọn **Inspect**.
2. DevTools mở ra và chọn sẵn thẻ `<h1>`. Bật **"Show accessibility tree"**. Cây thường hiện ngay, có heading đang được chọn.

Nếu vẫn không được: dùng cách xem thay thế

1. Tắt **"Show accessibility tree"** để quay về mã HTML.
2. Bấm chọn thẻ `<main id="main-content">` trong khung mã.
3. Ở tab **Accessibility** bên phải, mục **Accessibility Tree** sẽ hiện nhánh chứa phần tử đó, ví dụ:

```
RootWebArea "Vo Van Thien — Developer Portfolio"
  main
```

và mục *Computed properties* ghi **Role: main**.

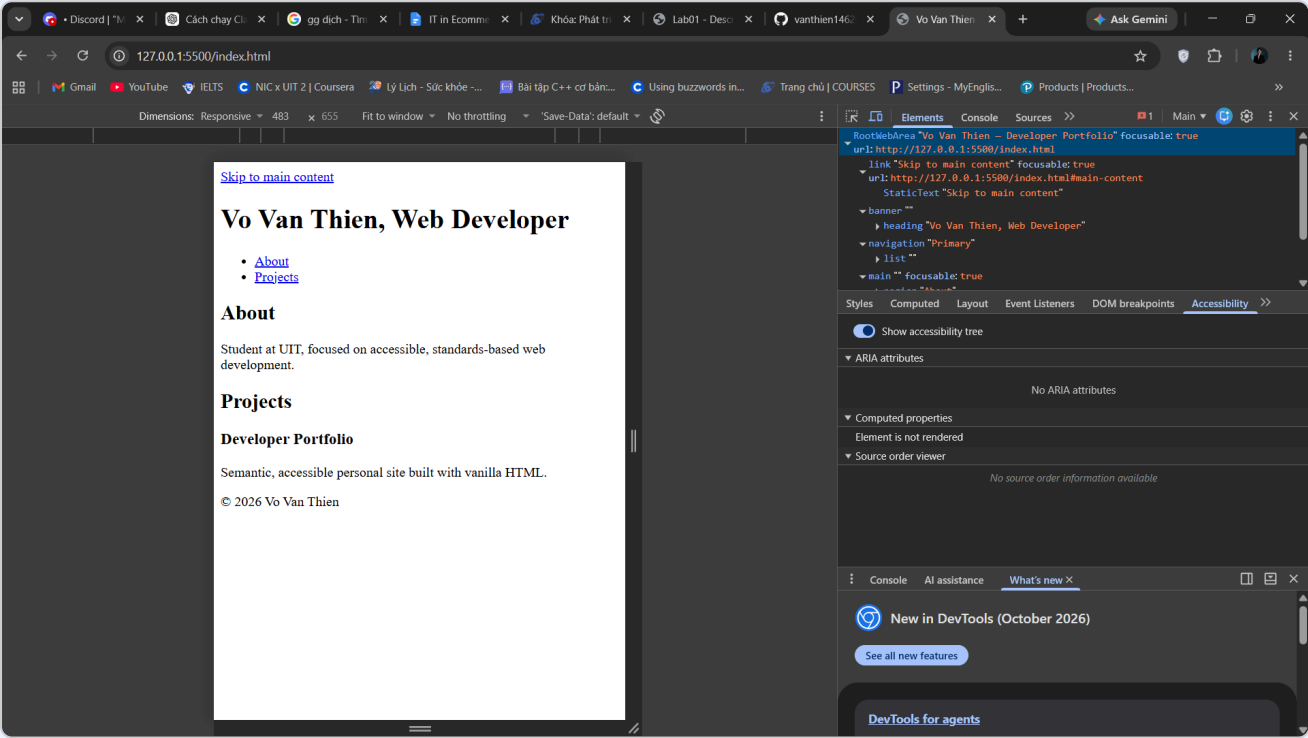
4. Làm tương tự với `<header>` (banner), `<nav>` (navigation "Primary"), mỗi `<section>` (region "About", region "Projects"), và `<footer>` (contentinfo).

Chụp một ảnh cho mỗi thẻ, hoặc chỉ chụp main và một section là đủ làm minh chứng.

Cách này không đẹp bằng cây toàn trang, nhưng vẫn chứng minh được từng landmark đúng role.

Làm xong bạn gửi ảnh, mình kiểm tra tiếp.

Bạn



Claude

Lần này cây **đúng rồi** ✅. Những gì đã thấy trong ảnh:

Trong cây	Ý nghĩa
RootWebArea "Vo Van Thien – Developer Portfolio", url 127.0.0.1:5500	Đúng trang của bạn
link "Skip to main content", url ...#main-content	Skip-link nằm đầu tiên và trở đúng đích
banner > heading "Vo Van Thien, Web Developer"	<header> và <h1>
navigation "Primary" > list	<nav aria-label="Primary">

Trong cây	Ý nghĩa
main focusable: true	<main> có tabIndex="-1" nên skip-link chuyển focus vào được
region "About" (bị che một nửa ở cuối)	<section> đã có tên

Chưa đủ để chụp: còn thiếu region "Projects" và contentinfo

Khung cây quá thấp nên phần dưới bị cắt. Làm cho khung rộng ra:

1. Bấm **Esc** (hoặc **×** ở khung "What's new") để đóng khung Console phía dưới.
2. **Kéo đường ngăn** giữa khung cây và hàng tab *Styles / Computed / ... / Accessibility* xuống dưới. Hoặc dùng cách nhanh hơn: chuyển DevTools sang dạng dọc toàn màn hình qua : → **Dock side** → **Undock**, hoặc dock sang phải cho cao hơn.
3. **Thu gọn các nhánh con** để cây ngắn lại: bấm **▾** ở link, banner, navigation để đóng chúng. Chỉ để mở main, để thấy 2 region bên trong.

Cây cần thấy đủ trong một ảnh:

```
RootWebArea "Vo Van Thien – Developer Portfolio"  
  link "Skip to main content"  
  banner  
  navigation "Primary"  
  main  
    region "About"  
    region "Projects"  
  contentinfo
```

Rồi chụp bằng **Win + Shift + S**.

⚠ Kiểm tra cái cờ đỏ "1" ở góc trên DevTools

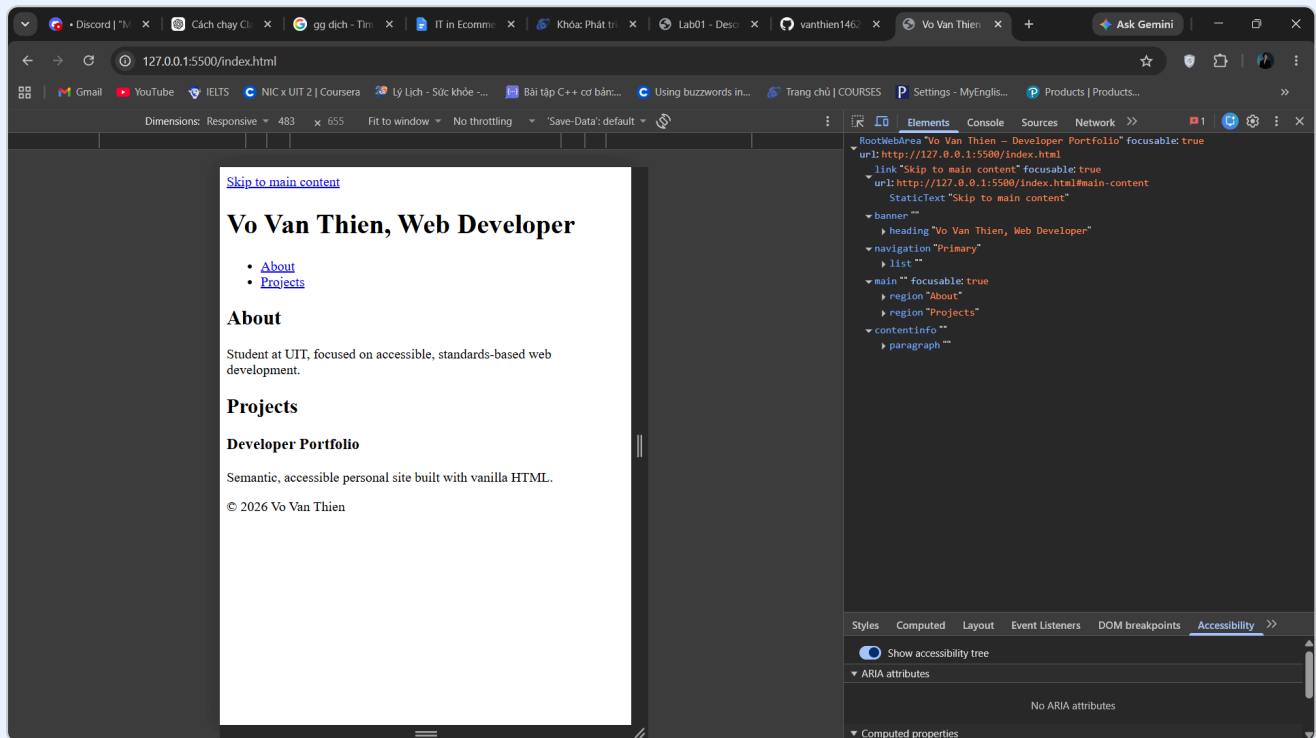
Ở góc trên DevTools, cạnh tab Sources, có biểu tượng  **1**: Console đang có **1 lỗi**. Bấm vào để xem. Nhiều khả năng đó là:

```
GET http://127.0.0.1:5500/favicon.ico 404 (Not Found)
```

Trình duyệt tự động tìm icon cho tab. Thư mục của bạn không có file đó, nên báo 404.

- **Nếu đúng là favicon:** lỗi vô hại và không phải do code của bạn. Exercise 1 không yêu cầu Console sạch, nên có thể để nguyên. Nhưng Exercise 2 yêu cầu "Zero console errors", nên khi tới đó bạn có thể thêm icon (một file `favicon.ico`, hoặc thêm `<link rel="icon" href="data:,">` vào `<head>`) trong một commit riêng.
- **Nếu là lỗi khác:** chụp hoặc dán nội dung lỗi lên đây để mình xem.

Bạn



Claude

Cây đã **đầy đủ và đúng hoàn toàn**. Ảnh này dùng làm minh chứng được

RootWebArea "Vo Van Thien – Developer Portfolio"	← đúng trang, 127.0.0.
link "Skip to main content" → #main-content	← skip-link đứng đầu
banner	← <header>
heading "Vo Van Thien, Web Developer"	← h1 duy nhất
navigation "Primary"	← <nav aria-label>
main focusable: true	← tabindex="-1" hoạt đ
region "About"	← <section> có tên
region "Projects"	
contentinfo	← <footer>

Cây khớp 100% với contract trong `TASK_DECOMPOSITION.md`, đủ 6 landmark.

Việc còn lại

1. Lưu ảnh (chụp bằng Win + Shift + S, ảnh đang mở như hiện tại là được) vào thư mục dự án, ví dụ docs/T-01-a11y-tree.png.

2. Commit riêng ảnh:

```
git add docs/T-01-a11y-tree.png
```

```
git diff --staged --stat
```

```
git commit -m "docs(a11y): add T-01 landmark tree verification"
```

3. Xem cái cờ đỏ  **1.** Biểu tượng này vẫn còn ở góc trên DevTools. Bấm vào để xem lỗi trong Console. Nếu là favicon.ico 404 thì vô hại với Exercise 1. Nếu là lỗi khác, dán lên đây để mình xem.

Xong 3 việc này là **Exercise 1 hoàn tất**: 4 commit sạch, có đặc tả, có code, có minh chứng kiểm chứng.

Bạn

chụp hình rồi giờ làm gì, vào vs code à?

Claude

Đúng rồi. Ảnh mới chụp đang nằm trong **clipboard**, giờ cần lưu nó vào thư mục dự án rồi commit. Làm như sau.

Bước 1: Tạo thư mục docs trong dự án

Trong VS Code (đang mở thư mục 24521671_VoVanThien_LAB1):

- Ở khung **Explorer** bên trái, rê chuột lên tên dự án và bấm biểu tượng **New Folder** ( +).
- Đặt tên là **docs** rồi Enter.

Bước 2: Lưu ảnh vào docs

Chọn **một** trong hai cách:

Cách A: từ thông báo vừa chụp

1. Ngay sau khi chụp, góc dưới bên phải màn hình hiện thông báo kèm ảnh nhỏ. **Bấm vào thông báo đó** để mở Snipping Tool.
2. Bấm **Ctrl+S** (hoặc biểu tượng ).
3. Trong hộp thoại lưu, đi tới OneDrive\Tài liệu\24521671_VoVanThien_LAB1\docs.
4. Đặt tên **T-01-a11y-tree.png**, kiểu file **PNG**, rồi bấm **Save**.

Cách B: nếu thông báo đã biến mất Windows 11 thường tự lưu ảnh chụp vào **Ảnh (Pictures)\Screenshots**.

1. Mở File Explorer, vào **Pictures** → **Screenshots** và tìm ảnh mới nhất.
2. Kéo thả ảnh đó vào thư mục **docs** trong khung Explorer của VS Code (hoặc copy rồi dán).
3. Đổi tên thành **T-01-a11y-tree.png** (chuột phải → Rename).

Mở ảnh trong VS Code để xem lại: phải thấy rõ thanh địa chỉ **127.0.0.1:5500** và cây Accessibility đầy đủ.

Bước 3: Commit ảnh

Trong VS Code, mở Terminal bằng **Ctrl + `** (phím nằm dưới phím Esc). Terminal sẽ mở đúng thư mục dự án. Chạy lần lượt:

```
git add docs/T-01-a11y-tree.png
```

```
git diff --staged --stat
```

Kết quả chỉ được có **1 dòng**, là docs/T-01-a11y-tree.png. Nếu đúng thì commit:

```
git commit -m "docs(a11y): add T-01 landmark tree verification"
```

Bước 4: Kiểm tra lần cuối

```
git log --oneline --stat
```

Bấm **q** để thoát nếu thấy dấu **:**. Bạn phải thấy **4 commit**:

1. docs(a11y): add T-01 landmark tree verification: chỉ có file ảnh

2. docs(spec): mark T-01 done
3. feat(html): semantic landmark tree
4. docs(spec): define component contracts & WBS table

Bạn dán kết quả git log lên đây, mình xác nhận lần cuối giúp bạn.

Bạn

ngoài ra tôi phải làm 1 file report riêng, tôi có thể để ảnh minh chứng lên file report luôn được không, khỏi dùng vs code

Claude

Được. Đề **không yêu cầu** để ảnh minh chứng trong repo Git. Slide 7 chỉ ghi "Verify Landmark Tree", tức là phải kiểm tra, còn để ảnh ở đâu không quy định. Nếu đã có file report riêng thì dán ảnh vào report là hợp lý nhất, và **bạn có thể bỏ qua hẳn Bước 1–3 ở trên** (tạo docs/, commit ảnh).

Cần lưu ý

1. **Phần Git vẫn giữ nguyên.** 3 commit bạn đã làm (docs(spec), feat(html), docs(spec): mark T-01 done) là bắt buộc, không thay được bằng report. Report chỉ là phần bổ sung.
2. **Nếu report là file Word/PDF nộp riêng** (không nằm trong repo): dán ảnh thẳng vào là xong, không đưng tới Git.
3. **Nếu muốn để file report vào repo:** commit nó **riêng**, ví dụ docs(report): add Lab 1 report, đừng gộp vào commit code.

Gợi ý nội dung phần Exercise 1 trong report

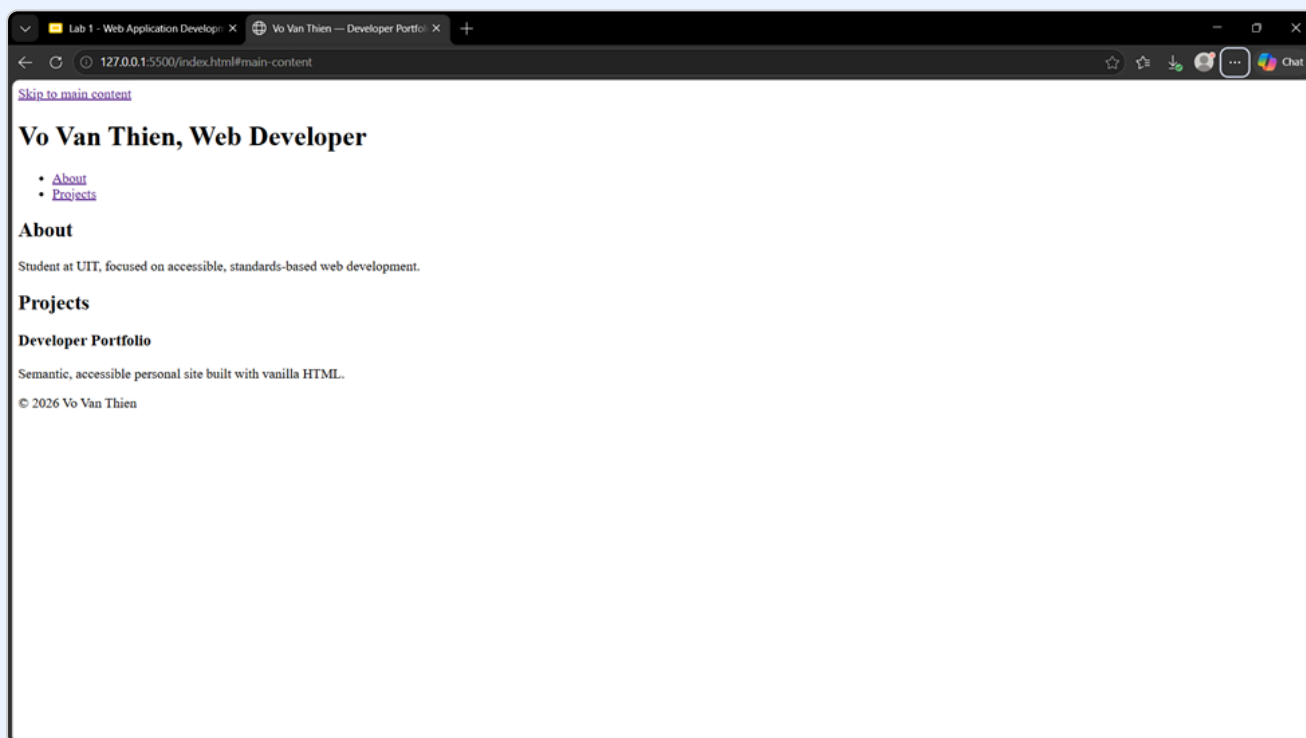
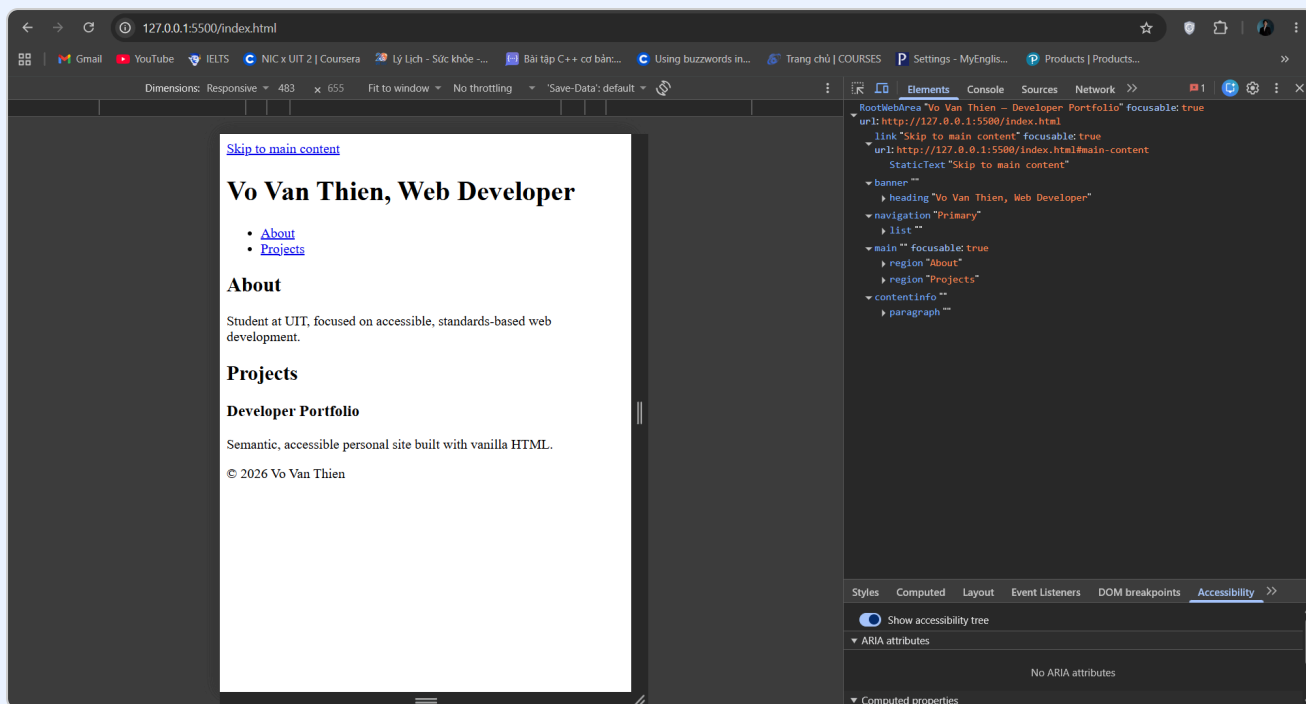
Mỗi ảnh nên có một câu chú thích ngắn nói ảnh chứng minh điều gì:

1. **Mục tiêu:** cây landmark semantic, 0 <div>, skip-link, 1 <h1>.
2. **Contract:** chép sơ đồ landmark từ TASK_DECOMPOSITION.md.
3. **Minh chứng:**
 - Ảnh trang chạy trên 127.0.0.1:5500.
 - **Ảnh cây Accessibility** (ảnh bạn vừa chụp): đủ banner, navigation, main, 2 region, contentinfo.

- Ảnh kết quả `git log --oneline --stat`: các commit atomic, không trộn CSS/JS.
 - (Nên có) Ảnh Console hiện `document.activeElement` là `<main>` sau khi Tab rồi Enter vào skip-link.
4. **Quyết định thiết kế:** `#main-content` thay cho `#main`, `tabindex="-1"`, không dùng role thừa, chưa link CSS (copy từ mục *Design Decisions*).
 5. **Sử dụng AI:** công cụ đã dùng, cách kiểm chứng, và lỗi AI mà bạn phát hiện (thiếu `tabindex="-1"`).

Bạn nên hỏi lại giảng viên report cần **định dạng gì** (Word, PDF hay Markdown) và **nộp ở đâu** (cùng repo hay nộp riêng), để làm đúng ngay từ đầu.

Bạn



tôi để 2 ảnh minh chứng này được rồi chứ

Claude

Ảnh 1 (cây Accessibility): đạt  Ảnh có thanh địa chỉ 127.0.0.1:5500, trang web, và cây đủ 6 landmark. Đây là minh chứng chính cho *Verification Gate*.

Ảnh 2 (sau khi bấm skip-link): dùng được nhưng còn yếu. Ảnh chỉ cho thấy URL đã đổi thành #main-content, tức là link nhảy đúng chỗ. Nó **không cho thấy focus đã vào**

<main>, mà đó mới là điểm chính của skip-link. Thêm nữa, ảnh này chụp bằng **Edge** (có nút "Chat" của Copilot), còn ảnh 1 chụp bằng Chrome.

Nên bổ sung

1. Ảnh git log --oneline --stat (quan trọng nhất). Exercise 1 chấm nặng phần commit atomic (STEP 4). Hai ảnh hiện tại không chứng minh được điều đó. Chụp terminal sau khi chạy lệnh, thấy đủ 3 commit và mỗi commit chỉ chứa file đúng loại.

2. Thay ảnh 2 bằng ảnh chứng minh focus (nên làm), chụp trong Chrome:

1. Mở trang, bấm vào thanh địa chỉ, rồi **Tab** để focus vào skip-link, rồi **Enter**.
2. Mở **Console** và gõ `document.activeElement`.
3. Chụp sao cho thấy cả URL `#main-content` lẫn kết quả `<main id="main-content" tabindex="-1">`.

Về cờ đỏ 🚩 1 trong ảnh 1

Ảnh 1 vẫn còn biểu tượng 🚩 1. Người chấm kỹ có thể để ý. Bấm vào xem là lỗi gì:

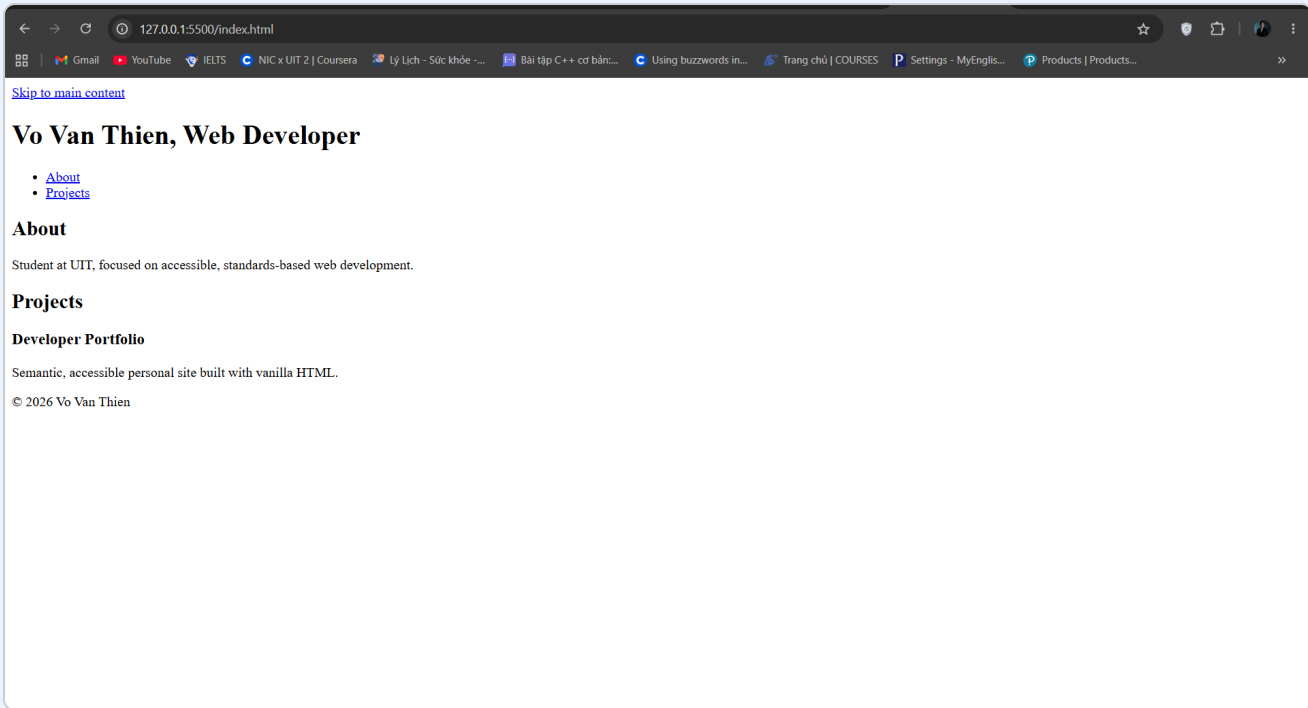
- Nếu là `favicon.ico 404`, ghi một dòng chú thích dưới ảnh: *"Lỗi 1 trong Console là favicon.ico 404 do trình duyệt tự yêu cầu icon; không liên quan đến T-01, sẽ xử lý ở T-02."*
- Nếu là lỗi khác, gửi mình xem trước khi nộp.

Gợi ý chú thích dưới mỗi ảnh

- **Hình 1:** Cây Accessibility trong Chrome DevTools: đủ banner, navigation "Primary", main, region "About", region "Projects", contentinfo; skip-link là phần tử đầu tiên.
- **Hình 2:** Sau khi Tab rồi Enter trên skip-link, focus chuyển vào `<main id="main-content">`.
- **Hình 3:** Lịch sử Git: spec trước code; commit HTML chỉ chứa `index.html`, không trộn CSS/JS.

Có đủ 3 ảnh này thì phần minh chứng Exercise 1 trong report là chắc chắn.

Bạn



thay lại bằng ảnh này nehs

Claude

Ảnh này **thay được cho ảnh "trang chạy trên Live Server"**: chụp bằng Chrome, thấy rõ 127.0.0.1:5500 và trang hiển thị đúng. Nhưng nó **không thay được ảnh minh chứng skip-link**, thậm chí còn yếu hơn ảnh Edge lúc này:

	Ảnh Edge trước	Ảnh Chrome này
Trình duyệt khớp đề	✗ Edge	✓ Chrome
URL đổi thành #main-content (link nhảy đúng chỗ)	✓	✗ chỉ là index.html
Focus vào <main>	✗	✗

Nhìn vào ảnh này, người chấm chỉ thấy trang hiển thị bình thường. **Không có gì chứng minh skip-link hoạt động.**

Cách chụp đúng ảnh skip-link (khoảng 1 phút)

1. Trong tab Chrome này, bấm **F12** và mở tab **Console**.
2. Bấm vào **vùng trắng của trang**, rồi bấm **F5** để tải lại.
3. Bấm **Tab** một lần. Skip-link "Skip to main content" sẽ có viền focus.
4. Bấm **Enter**. URL đổi thành ...index.html#main-content.

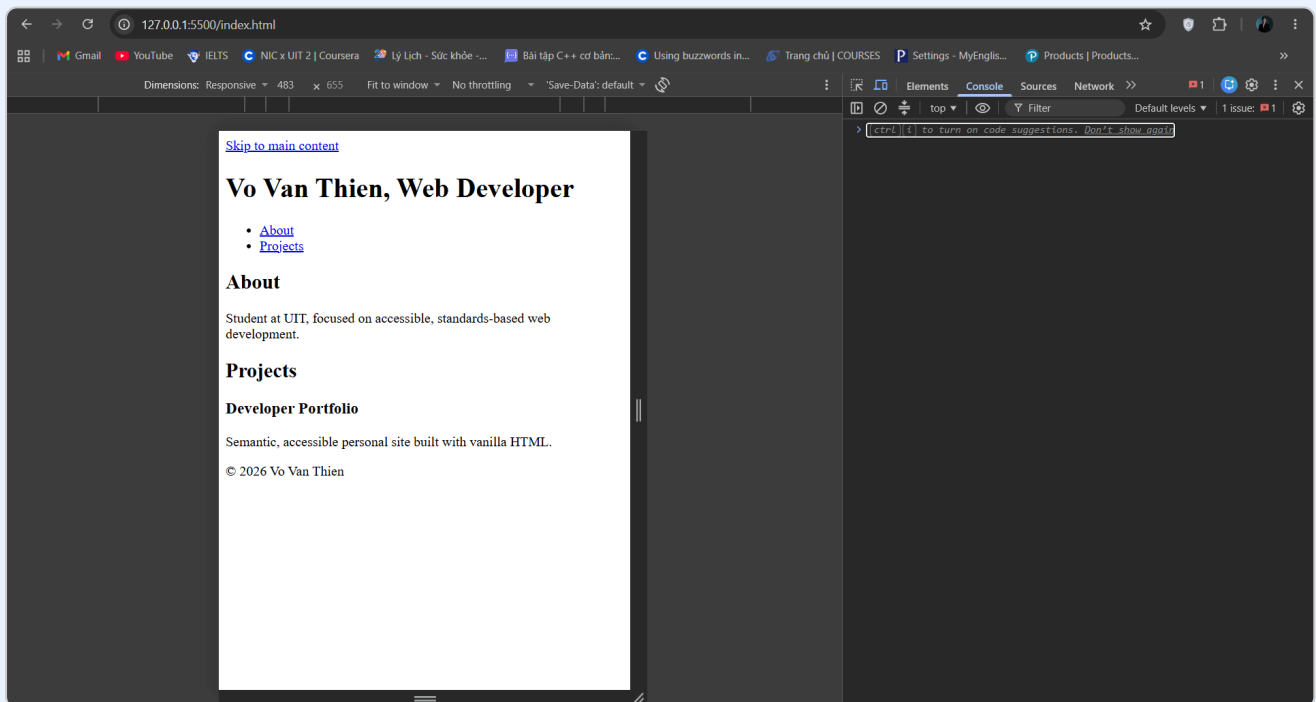
5. Bấm vào ô gõ lệnh của Console, gõ `document.activeElement` rồi Enter.
6. Console phải in ra `<main id="main-content" tabindex="-1">...</main>`.
7. Chụp bằng **Win + Shift + S**, sao cho thấy cả **URL #main-content** lẫn **kết quả trong Console**.

Lưu ý ở bước 5: bấm vào Console sẽ làm focus chuyển sang DevTools. Nhưng `document.activeElement` vẫn trả về phần tử đang được focus *trong trang*, nên kết quả vẫn đúng là `<main>`.

Bộ ảnh cho report

1. **Ảnh này:** trang chạy trên Live Server (Chrome). ✓
2. **Ảnh cây Accessibility** (đã có). ✓
3. **Ảnh skip-link + Console `document.activeElement`.** Cần chụp lại theo các bước trên.
4. **Ảnh git log --oneline --stat.** Chưa có, đây là ảnh quan trọng nhất.

Bạn



này hả

Claude

Chưa phải. Đây mới là bước mở Console. Ảnh hiện tại còn thiếu hai thứ:

- URL vẫn là `index.html`, **chưa có #main-content**, tức là skip-link chưa được bấm.

- Console **trống**, chưa gõ lệnh nào.

Làm tiếp từ màn hình này

1. Bấm vào **vùng trắng của trang** (bên trái, ví dụ chỗ trống dưới dòng © 2026).
2. Bấm **F5** để tải lại trang (DevTools vẫn để mở).
3. Bấm **Tab** một lần. "Skip to main content" sẽ có **viền focus** bao quanh.
4. Bấm **Enter**. URL đổi thành `127.0.0.1:5500/index.html#main-content`.
5. Bấm vào **dòng có dấu >** trong Console (chỗ đang hiện `"ctrl i to turn on code suggestions"`).
6. Gõ đúng lệnh sau rồi Enter:

```
document.activeElement
```

7. Console in ra `<main id="main-content" tabindex="-1">...</main>`.
8. Chụp bằng **Win + Shift + S**, sao cho thấy **URL có #main-content** và **dòng kết quả <main ...>** trong Console.

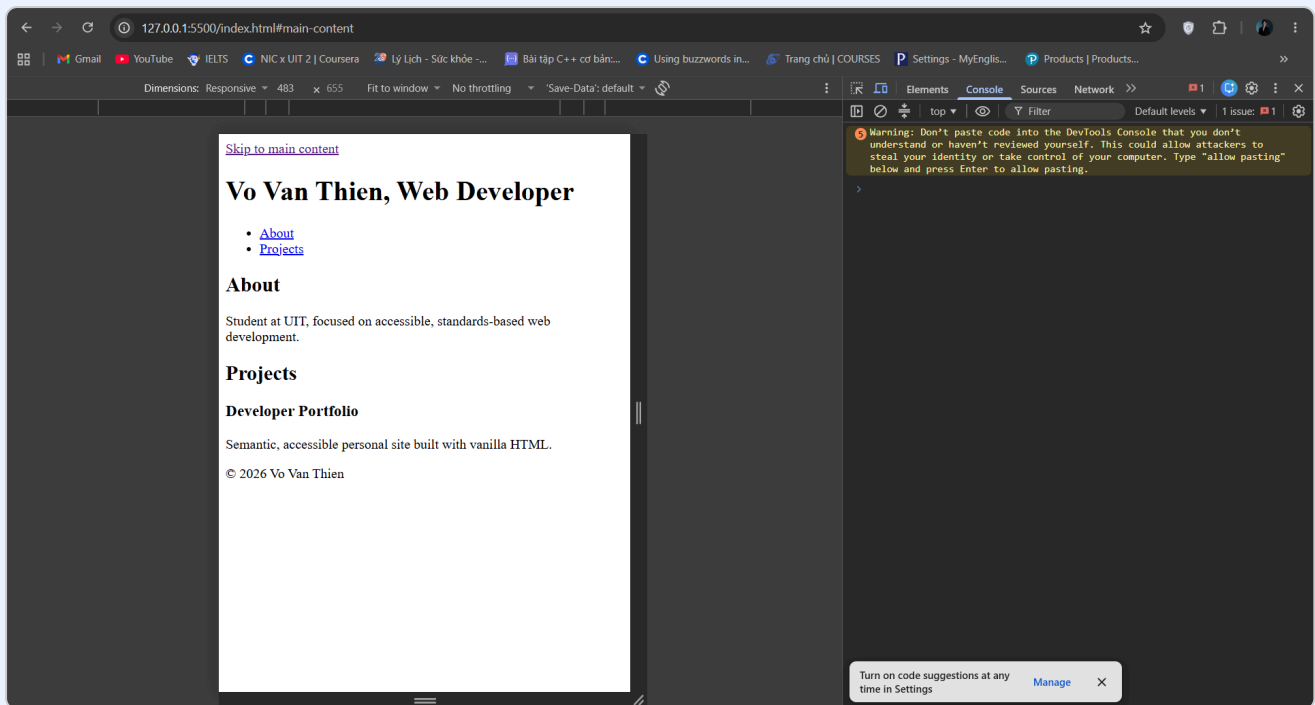
Nếu bước 3 không thấy viền focus: có thể focus đang ở trong DevTools. Bấm lại vào vùng trắng của trang rồi thử Tab lần nữa.

Nếu bước 7 ra <body> thay vì <main>: nghĩa là focus chưa vào main. Làm lại từ bước 1, và nhớ bấm Tab rồi Enter **trước khi** bấm vào Console.

Về "1 issue"

Góc phải thanh Console ghi **"1 issue"**. Đây là **Issue** (gợi ý hoặc cảnh báo của Chrome), không phải lỗi JavaScript. Bấm vào chữ **"1 issue"** để xem nội dung. Thường đó là những gợi ý vô hại, không liên quan đến code T-01. Nếu muốn chắc chắn, chụp nội dung issue gửi mình xem.

Bạn



Claude

Bước skip-link đã đúng: URL đã thành **#main-content** ✓

Cảnh báo vàng xuất hiện vì bạn **dán (paste)** lệnh vào Console. Chrome chặn việc dán vào Console để chống lừa đảo kiểu *self-XSS*: kẻ xấu dụ người dùng dán đoạn mã độc vào Console để chiếm tài khoản. Số **5** trong vòng tròn nghĩa là bạn đã thử dán 5 lần và đều bị chặn.

Cách xử lý: tự gõ lệnh, đừng dán

1. Bấm **X** để đóng thông báo nhỏ "Turn on code suggestions" ở góc dưới.
2. Bấm vào dòng có dấu **>** trong Console.
3. **Tự gõ từng chữ** (không Ctrl+V):

```
document.activeElement
```

4. Bấm **Enter**. Kết quả phải là `<main id="main-content" tabindex="-1">...</main>`.
5. Chụp bằng **Win + Shift + S**, sao cho thấy URL `#main-content` và dòng kết quả.

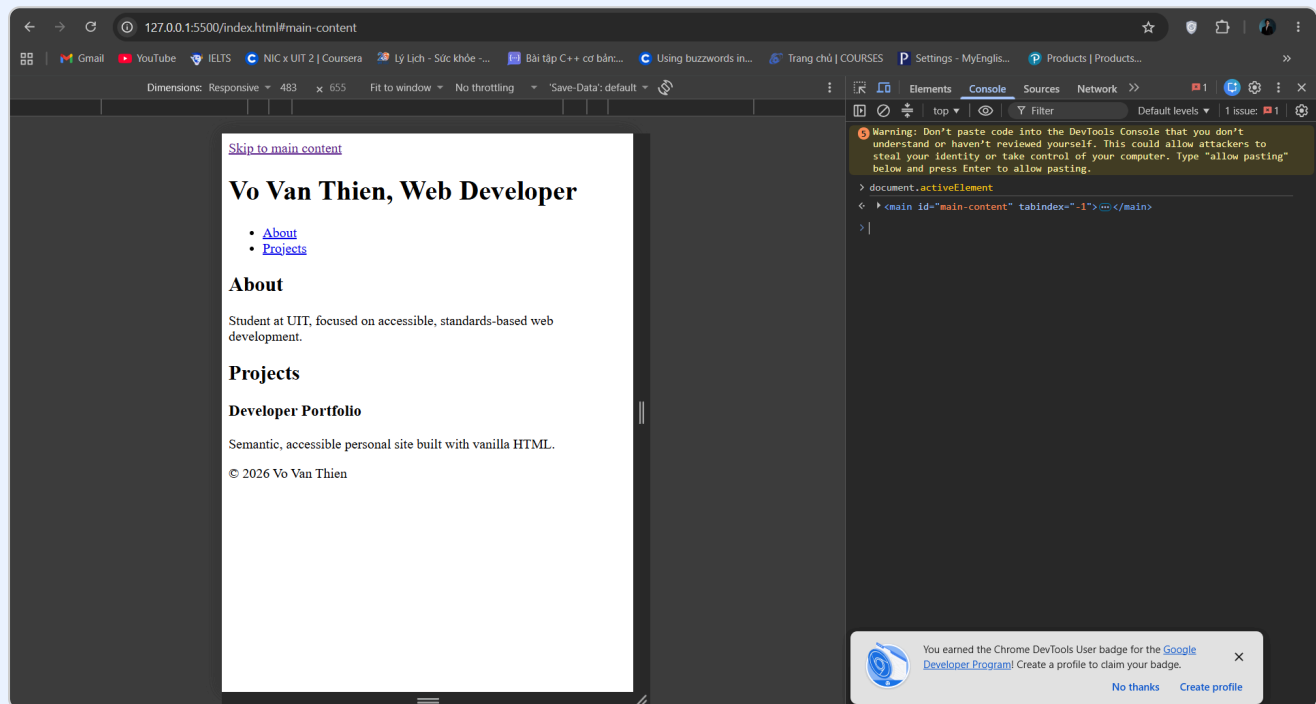
Cảnh báo vàng nằm trong ảnh cũng không sao. Nếu muốn ảnh gọn hơn, bấm biểu tượng **🗑️ (Clear console)** ở đầu thanh Console để xóa trước, rồi mới gõ lệnh.

Có nên gõ "allow pasting" không?

Không cần. Lệnh này chỉ có 22 ký tự, tự gõ nhanh hơn. Gõ allow pasting sẽ **tắt lớp bảo vệ đó** cho trình duyệt. Với lệnh bạn hiểu rõ như thế này thì không nguy hiểm, nhưng tốt nhất đừng tập thói quen tắt nó.

Bạn có thể ghi chi tiết này vào phần Security của report. Nó liên quan trực tiếp tới chủ đề XSS trong Lab (slide 2, 21).

Bạn



Claude

Đây đúng là ảnh minh chứng cho skip-link ☒

- URL: 127.0.0.1:5500/index.html#main-content, tức là skip-link đã nhảy đúng đích.
- Console: document.activeElement trả về <main id="main-content" tabindex="-1">, tức là **focus đã vào <main>**.

Ảnh này chứng minh đúng tiêu chí "Tab lần đầu focus vào skip-link; Enter chuyển focus vào <main>".

Để ảnh gọn hơn (không bắt buộc): bấm X để đóng thông báo "You earned the Chrome DevTools User badge" ở góc dưới, rồi chụp lại. Cảnh báo vàng phía trên để nguyên cũng được.

Bộ ảnh cho report

1.  Trang chạy trên Live Server (Chrome).
2.  Cây Accessibility đủ 6 landmark.
3.  Skip-link: URL #main-content và document.activeElement là <main>.
4.  **Ảnh git log --oneline --stat.** Đây là ảnh duy nhất còn thiếu.

Để chụp ảnh 4: mở Terminal trong VS Code (**Ctrl + `**) hoặc PowerShell tại thư mục dự án, rồi chạy:

```
git log --oneline --stat
```

Chụp sao cho thấy đủ 3 commit và file trong từng commit. Nếu thấy dấu : ở cuối thì chụp xong mới bấm **q** để thoát.

Có ảnh này nữa là **phần minh chứng Exercise 1 hoàn chỉnh**.

Bạn

```
PS C:\Users\USER\OneDrive\Tài liệu\24521671_VoVanThien_LAB1> git log --oneline --stat
b2c9ecf (HEAD -> main) docs(spec): mark T-01 done TASK_DECOMPOSITION.md | 55
+++++----- 1 file changed, 35
insertions(+), 20 deletions(-) 4622638 feat(html): semantic landmark tree index.html | 42
+++++ 1 file changed, 42
insertions(+) 7231598 docs(spec): define component contracts & WBS table
TASK_DECOMPOSITION.md | 38 ++++++
project-rules.md | 7 ++++++ :
```

Claude

Kết quả này đạt yêu cầu  Có 3 commit, đúng thứ tự spec trước code, và mỗi commit chỉ chứa đúng loại file của nó.

Đưa vào report

Có hai cách, chọn một:

- **Chụp ảnh** terminal đang hiện kết quả này (Win + Shift + S).
- **Dán dạng chữ** vào report, đặt trong khung code hoặc font Consolas. Cách này rõ hơn ảnh, người chấm còn copy được.

Mẹo để không bị cắt mất dòng cuối: dấu : cuối kết quả làm dòng 2 files changed, 45 insertions(+) của commit đầu tiên bị ẩn. Chạy lệnh sau để Git in hết ra một lần, không dừng lại:

```
git --no-pager log --oneline --stat
```

Kiểm tra cuối (nếu chưa làm)

Lịch sử Git chỉ cho thấy commit chứa file nào, chưa cho thấy trong index.html có gì. Chạy thêm:

```
git show 4622638 | Select-String -Pattern '<div|style=|<style|<script|onc
```

Không in ra gì là commit HTML sạch hoàn toàn. Bạn có thể ghi kết quả này vào report như một bước kiểm chứng.

Tổng kết Exercise 1

Yêu cầu	Trạng thái
STEP 1: khai báo T-01 trong TASK_DECOMPOSITION.md	✓
STEP 2: contract cây landmark, 0 <div>	✓
STEP 3: skip-link hoạt động	✓ (ảnh activeElement là <main>)
STEP 4: commit atomic feat(html)	✓
Verification gate: cây Accessibility	✓ (ảnh đủ 6 landmark)
Không trộn CSS với HTML	✓

Exercise 1 đã hoàn tất. Khi nộp, nhớ kèm cả thư mục ẩn .git (nếu nộp file nén) hoặc git push (nếu nộp qua GitHub), để giảng viên thấy được lịch sử commit.

Bạn

nếu git push thì trong report chỉ cần kèm theo 3 tấm ảnh thôi ha

Claude

Gần đúng. Nếu đã git push, **lịch sử commit nằm sẵn trên GitHub**, giảng viên tự xem được, nên ảnh git log không còn bắt buộc. Nhưng report cần thêm vài thứ ngoài 3 ảnh.

Report nên có

1. **Link repo GitHub.** Bắt buộc, vì đây là thứ thay cho ảnh git log. Kiểm tra repo để **Public**, hoặc đã mời giảng viên vào nếu là Private. Mở link bằng cửa sổ ẩn danh để chắc người khác xem được.
2. **3 ảnh minh chứng, kèm một câu chú thích cho mỗi ảnh:**
 - Trang chạy trên Live Server.
 - Cây Accessibility đủ 6 landmark.
 - Skip-link: URL #main-content và activeElement là <main>.
3. **Vài dòng giải thích ngắn** (có thể copy từ TASK_DECOMPOSITION.md):
 - Quyết định thiết kế: #main-content thay cho #main, tabindex="-1", không dùng role thừa, chưa link CSS.
 - Cách sử dụng AI và lỗi AI đã phát hiện.

Ảnh git log không bắt buộc nữa, nhưng **kèm thêm vẫn tốt**: người chấm thấy ngay trong report, không cần mở GitHub.

Trước khi push

Nếu repo chưa có trên GitHub:

1. Tạo repo mới trên GitHub, **không** tích *Add README*, để không phải gộp lịch sử.
2. Chạy:

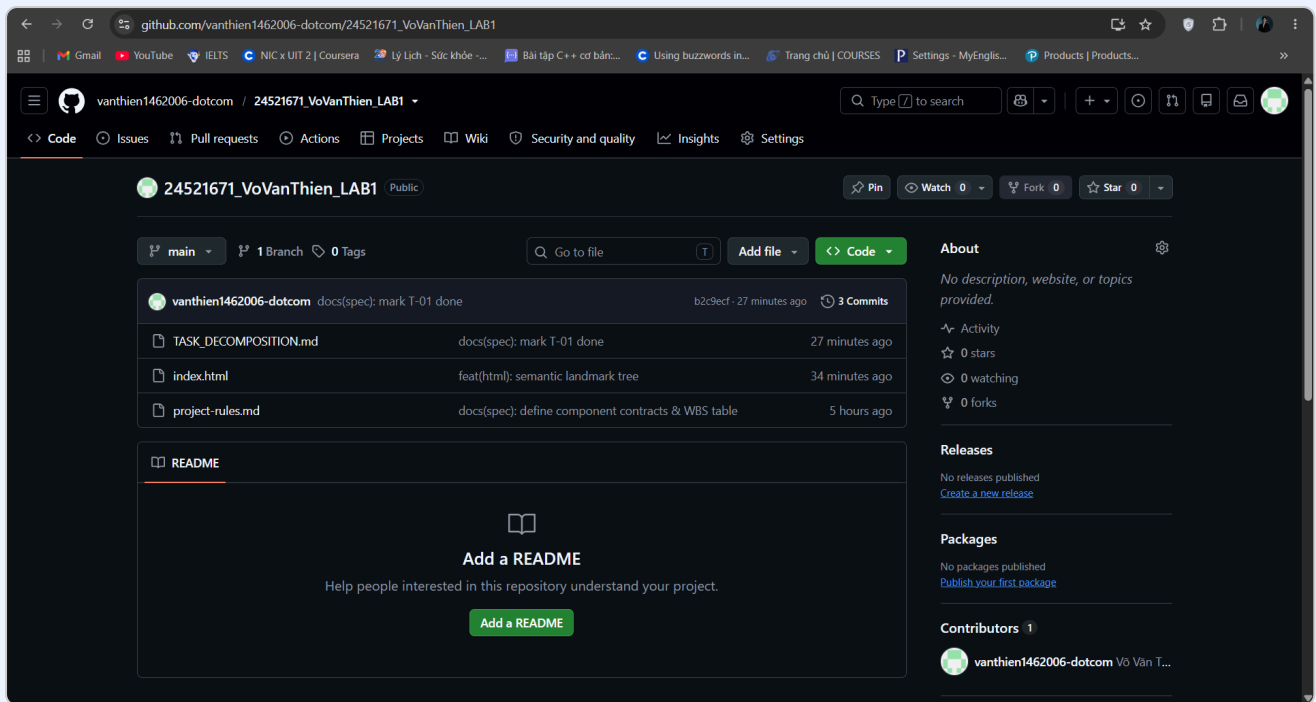
```
git remote add origin https://github.com/<ten-github>/<ten-repo>.git
```

```
git push -u origin main
```

3. Mở repo trên GitHub, vào mục **Commits** để chắc chắn đủ 3 commit.

Đây là gợi ý chung. Báo cáo cần định dạng gì, nộp ở đâu thì nên **xem lại yêu cầu của giảng viên**.

Bạn



thành công rồi chứ

Claude

Push **thành công** ✓

- Repo **Public**, nên giảng viên xem được.
- Có **3 Commits**, khớp với lịch sử trên máy.
- Có đủ 3 file. Mỗi file hiện đúng commit gần nhất đã sửa nó: `index.html` thuộc `feat(html)`, hai file `.md` thuộc các commit `docs(spec)`.

Link ghi vào report:

https://github.com/vanthien1462006-dotcom/24521671_VoVanThien_LAB1

⚠️ Đừng bấm "Add a README" trên GitHub

Nút xanh đó sẽ **tạo commit trực tiếp trên GitHub**. Khi đó lịch sử trên GitHub có thêm một commit mà máy bạn không có. Lần `git push` tiếp theo (ở Exercise 2) sẽ **bị từ chối**, và bạn phải `git pull` để gộp, rất dễ làm lịch sử rối.

Nếu muốn có README, hãy tạo **trên máy** rồi commit riêng:

```
git add README.md
```

```
git commit -m "docs: add README"
```

```
git push
```

README không bắt buộc cho Exercise 1, nên bỏ qua cũng được.

Exercise 1 đã hoàn tất: code, commit, minh chứng và repo public đều xong. Khi làm Exercise 2, cứ commit theo từng sub-task (T-02A, T-02B, T-02C) rồi `git push` như bình thường.

Bạn

/expert

Claude

Lệnh **/expert** **không có** trong phiên này nên chưa chạy. Nếu bạn muốn dùng nó, có thể thêm dưới dạng plugin của tổ chức hoặc skill của dự án. Mình không chắc cách cài cụ thể nên không hướng dẫn nữa.

Bạn muốn mình làm gì tiếp? Ví dụ:

- **Rà soát Exercise 1** theo góc nhìn người chấm, tìm chỗ có thể bị trừ điểm khi vấn đáp.
- **Hướng dẫn Exercise 2** (T-02A tokens và reset, T-02B grid, T-02C dark mode) theo cùng quy trình atomic commit.
- **Soạn dàn ý report** cho Exercise 1.
- **Luyện vấn đáp:** mình đóng vai giảng viên và hỏi bạn về code.

Bạn chỉ cần nói phần nào, mình làm tiếp.

Bạn

Bạn có thể nào xuất file link trò chuyện này được không